

Facultad
de Informática

UMU

Universidad de Murcia

Grado en Ingeniería Informática

Soft Computing, Explícate

Autor:
Jorge Montesinos Bravo
Tutor:
Mercedes Valdés Vela

28 de mayo de 2026

Convocatoria de Junio

Agradecimientos

Me gustaría dedicar este Trabajo Fin de Máster a todas las personas que me han ayudado en este proceso.

En primer lugar, a mi tutora del TFM, Mercedes Valdés Vela, por brindarme ayuda siempre que la he necesitado, tanto en este trabajo, como durante la carrera y el máster.

También me gustaría agradecer a mi familia, en especial a mis padres y mi hermana, por ser de gran apoyo durante toda la carrera, y sobre todo durante este último año.

A Noelia, por ser uno de los pilares que ha estado conmigo siempre, sirviéndome de gran ayuda tanto en lo personal como en lo profesional, y por ayudarme a seguir adelante hasta cuando ni yo mismo pensaba en poder seguir.

A Claudia, Javi, Marta, Rocío y Ainhoa, por animarme cuando más lo necesitaba y ayudarme a saber cuándo es necesario parar un poco para seguir con más fuerza.

Por último, a Javier, Vania, Alfonso, Carlos, Miguel y María, por ayudarme siempre, incondicionalmente, y hacerme saber que no es necesaria una amistad de toda la vida para sentir que esta puede ser igual de importante que cualquier otra.

Índice general

1. Introducción **7**

2. Objetivos y metodología **8**
2.1. Objetivos 8

3. Estado de la cuestión **9**
3.1. IA Explicable (XAI) 9
3.1.1. Tipos de XAI 11
3.1.2. Shap 13
3.1.3. LIME 16
3.1.4. Estado actual de la IA Explicable 20
3.2. Lógica Difusa y Soft Computing 25
3.2.1. Conjuntos Difusos y Funciones de Pertenencia 25
3.2.2. Variables Lingüísticas 26
3.2.3. Sistemas de Inferencia Difusos 26
3.2.4. Soft Computing 29
3.2.5. Estado actual de la Lógica Difusa y el Soft Computing 31
3.3. Propuestas de combinación entre XAI y Soft Computing 33

4. Implementación **34**
4.1. Contextualización 34
4.1.1. Enfoques de Explicabilidad Difusa Propuestos 34
4.1.2. Justificación del uso de Reglas Difusas frente a un Explicador Lineal . . . 35
4.1.3. Entrenamiento del modelo de caja negra 35
4.2. Diseño e implementación del dataset sintético 36
4.3. Fuzzy-LIME 36
4.3.1. Extracción de reglas 37
4.3.2. Fuzzificación del vocabulario 38
4.3.3. Traductor Lingüístico 38
4.3.4. Agregación global de reglas difusas 39
4.3.5. Reducción de la Base de Reglas 39
4.3.6. Inferencia Difusa 40
4.3.7. Ejemplo de uso 41
4.4. API (Librería) de Fuzzy-LIME 41
4.4.1. Creación de la librería 42
4.4.2. Uso de la librería 42
4.5. FuzzyRank-LIME 44
4.5.1. Fundamentación y Metodología de Implementación 44
4.5.2. Justificación del uso del Enfoque Lingüístico 45
4.6. SC-LIME 45

4.6.1. Limitaciones de la Selección Submodular Convencional 45
4.6.2. Fundamentación y Metodología de SC-LIME 46
4.6.3. Justificación del uso del Análisis Basado en Prototipos 46

5. Experimentación **47**
5.1. Evaluación de Fuzzy-LIME 47
5.1.1. Análisis Local: Explicación de una instancia individual 47
5.1.2. Análisis Global: Comprensión general del modelo 48
5.1.3. Evaluación con un dataset distinto 50
5.1.4. Evaluación con un dataset real 53
5.2. Evaluación de FuzzyRank-LIME 56
5.2.1. Resultados con el dataset original 56
5.2.2. Resultados sobre un dataset real 59
5.3. Evaluación de SC-LIME 62
5.3.1. Análisis sobre el Conjunto de Datos Original 62
5.3.2. Análisis sobre el nuevo Conjunto de Datos 63
5.3.3. Análisis sobre un Conjunto de Datos real 64

6. Conclusiones y vías futuras **66**

Bibliografía **68**

Apéndice **70**

Resumen

En este Trabajo de Fin de Máster se aborda el problema de la explicabilidad en los sistemas de Inteligencia Artificial, especialmente en aquellos modelos predictivos complejos cuya lógica interna resulta difícil de interpretar para los usuarios. En el contexto de la Inteligencia Artificial Explicable (XAI), se estudian algunas de las limitaciones que presentan las técnicas actuales para proporcionar explicaciones comprensibles, estables y cognitivamente accesibles.

Antes del desarrollo de las técnicas modernas de XAI, algunos de los modelos de inteligencia artificial considerados más interpretables eran los sistemas basados en reglas difusas, capaces de representar el conocimiento mediante expresiones lingüísticas del tipo SI–ENTONCES. Estos sistemas forman parte del paradigma del *Soft Computing*, un conjunto de metodologías orientadas al tratamiento de la incertidumbre, la imprecisión y la aproximación, entre las que se incluyen la Lógica Difusa, los sistemas neuro-difusos o ciertas técnicas de clustering.

A partir de esta relación natural entre interpretabilidad y razonamiento aproximado, surge la pregunta de si el *Soft Computing* (y, de manera más concreta, los sistemas de reglas difusas) puede aportar nuevos enfoques al ámbito de la XAI. Con este objetivo, el trabajo se centra en el estudio de una de las herramientas más conocidas de explicación local, LIME, analizando cómo pueden contribuir tanto los sistemas de inferencia difusa como los números difusos a mejorar algunos de sus aspectos interpretativos y de estabilidad.

Para ello, se desarrolla una prueba de concepto basada en tres aproximaciones complementarias. En primer lugar, se propone **Fuzzy-LIME**, donde el modelo lineal subrogado de LIME se sustituye por un Sistema de Inferencia Difuso capaz de generar reglas lingüísticas comprensibles. En segundo lugar, se plantea **FuzzyRank-LIME**, orientado a estabilizar las explicaciones mediante iteraciones sucesivas y técnicas de clustering difuso. Finalmente, se introduce **SC-LIME**, que emplea *Subtractive Clustering* para seleccionar automáticamente explicaciones globales representativas sin necesidad de fijar previamente su número.

El propósito de estas propuestas no es establecer una solución definitiva, sino explorar si existe una posible sinergia entre el *Soft Computing* y la XAI. Los resultados obtenidos, tanto sobre funciones sintéticas como sobre conjuntos de datos reales, sugieren que esta combinación puede resultar prometedora para reducir la carga cognitiva de las explicaciones y representar de manera más natural la lógica subyacente de modelos complejos, aunque manteniendo ciertas limitaciones inherentes al uso de representaciones lingüísticas frente a valores numéricos precisos.

Como resultado adicional del trabajo, se ha desarrollado una API interactiva en Python que integra la herramienta Fuzzy-LIME para poder ajustar los parámetros de entrenamiento del modelo subrogado y mejorar su interpretabilidad visual. En conjunto, el estudio pretende servir como una primera aproximación exploratoria sobre la posible complementariedad entre ambas disciplinas y dejar abiertas futuras líneas de investigación más profundas.

Abstract

This Master's Thesis addresses the problem of explainability in Artificial Intelligence systems, especially in complex predictive models whose internal logic is difficult for users to interpret. Within the context of Explainable Artificial Intelligence (XAI), some of the limitations of current techniques for providing understandable, stable, and cognitively accessible explanations are examined.

Before the emergence of modern XAI techniques, some of the Artificial Intelligence models considered most interpretable by humans were those based on fuzzy rule systems, capable of representing knowledge through linguistic IF–THEN expressions. These systems are part of the Soft Computing paradigm, a set of methodologies aimed at handling uncertainty, imprecision, and approximation, including Fuzzy Logic, neuro-fuzzy systems, and certain clustering techniques.

From this natural relationship between interpretability and approximate reasoning, the question arises as to whether Soft Computing (and, more specifically, fuzzy rule-based systems) can contribute new approaches to the field of XAI. With this objective, the thesis focuses on the study of one of the best-known local explanation tools, LIME, analyzing how both fuzzy inference systems and fuzzy numbers may help improve some of its interpretability and stability aspects.

To this end, a proof of concept based on three complementary approaches is developed. First, **Fuzzy-LIME** is proposed, where LIME's surrogate linear model is replaced by a Fuzzy Inference System capable of generating understandable linguistic rules. Second, **FuzzyRank-LIME** is introduced, aimed at stabilizing explanations through iterative executions and fuzzy clustering techniques. Finally, **SC-LIME** is presented, employing Subtractive Clustering to automatically select the most representative global explanations without the need to predefine their number.

The purpose of these proposals is not to establish a definitive solution, but rather to explore whether a possible synergy exists between Soft Computing and XAI. The results obtained, both on synthetic functions and real-world datasets, suggest that this combination may be promising for reducing the cognitive load of explanations and representing the underlying logic of complex models in a more natural way, although certain limitations inherent to linguistic representations compared to precise numerical values still remain.

As an additional outcome of this work, an interactive Python API integrating the Fuzzy-LIME tool has been developed to enable the adjustment of the surrogate model training parameters and improve its visual interpretability. Overall, this study aims to serve as an initial exploratory approach to the possible complementarity between both disciplines and to open the door to future, more in-depth lines of research.

Capítulo 1

Introducción

En los últimos años, la Inteligencia Artificial ha experimentado un crecimiento muy significativo debido, principalmente, al aumento de la capacidad de cálculo y a la disponibilidad masiva de datos. Este avance ha permitido desarrollar modelos cada vez más complejos y precisos, especialmente en ámbitos relacionados con el Aprendizaje Automático y el Deep Learning. Sin embargo, esta mejora en el rendimiento predictivo ha venido acompañada de una pérdida progresiva de interpretabilidad. Muchos de estos modelos funcionan como cajas negras, capaces de generar resultados con gran precisión, pero sin ofrecer explicaciones claras sobre el razonamiento seguido para alcanzar una determinada decisión.

Esta falta de explicaciones supone un problema importante en contextos donde resulta necesario comprender, validar o justificar las predicciones realizadas por un sistema inteligente. Como consecuencia, en los últimos años ha surgido un creciente interés por la Inteligencia Artificial Explicable (XAI), cuyo objetivo consiste en desarrollar métodos capaces de aportar interpretaciones comprensibles sobre el comportamiento de modelos complejos.

Frente a esta tendencia hacia modelos altamente opacos, existen otros paradigmas de la Inteligencia Artificial que históricamente han estado ligados a la interpretabilidad. Entre ellos destacan la Lógica Difusa y, de forma más general, el paradigma del *Soft Computing*. Estos enfoques se caracterizan por representar el conocimiento mediante reglas lingüísticas y mecanismos de razonamiento aproximado, permitiendo construir sistemas más cercanos a la forma en la que las personas describen y comprenden determinados problemas. Aunque muchas de estas técnicas han perdido protagonismo frente al auge del *Deep Learning*, continúan ofreciendo una ventaja relevante desde el punto de vista de la transparencia y la comprensión de los modelos.

A partir de esta situación surge la cuestión principal que motiva este trabajo: estudiar si ambos paradigmas pueden complementarse entre sí. En concreto, se pretende analizar si algunas técnicas procedentes de la Lógica Difusa y el *Soft Computing* pueden contribuir a mejorar ciertos aspectos relacionados con la interpretabilidad dentro del ámbito de la XAI. El propósito de este trabajo no es plantear una solución definitiva, sino realizar una prueba de concepto que permita valorar si existe una posible sinergia entre ambas disciplinas.

Para ello, este Trabajo de Fin de Máster se estructura de la siguiente manera. En primer lugar, se presentan los objetivos y la metodología seguida durante el desarrollo del trabajo. Posteriormente, se introducen los conceptos fundamentales relacionados con los dos paradigmas utilizados, abordando tanto las bases de la Inteligencia Artificial Explicable como los fundamentos de la Lógica Difusa y el *Soft Computing*. A continuación, se realiza una revisión del estado actual de ambas disciplinas y de algunas investigaciones relacionadas. Finalmente, se expone la implementación desarrollada, junto con los resultados obtenidos, las conclusiones derivadas del estudio y las posibles vías futuras de investigación.

Capítulo 2

Objetivos y metodología

2.1. Objetivos

El objetivo principal de esta propuesta es explorar si existe una **sinergia** entre las técnicas clásicas del *Soft Computing* y las herramientas actuales de explicabilidad de modelos opacos. En otras palabras, queremos reivindicar el posible papel de la Lógica Difusa en una época dominada por el *Deep Learning*, donde el aumento del rendimiento predictivo suele venir acompañado de una pérdida de transparencia, analizando si los modelos basados en reglas difusas pueden aportar mecanismos de interpretabilidad para comprender mejor el comportamiento de estos modelos de "caja negra".

Para conseguir esta meta, llevaremos a cabo las siguientes tareas:

- Realizar una revisión del estado de la cuestión tanto en el ámbito de la Inteligencia Artificial Explicable (XAI) como en el paradigma del *Soft Computing*. Para ello, se introducirán primero los fundamentos teóricos necesarios de ambas disciplinas y, posteriormente, se analizarán algunas de las investigaciones y enfoques más relevantes desarrollados actualmente en cada una de ellas.
- Analizar, a partir de las técnicas revisadas en ambos paradigmas, qué posibles combinaciones pueden establecerse entre métodos de *Soft Computing* y herramientas de XAI, para llevar a cabo una prueba de concepto sobre si esta fusión es capaz de conseguir la sinergia que buscamos.
- Implementar diferentes maneras de combinar las técnicas escogidas de *Soft Computing* y XAI y explorar las posibles sinergias.
- Evaluar y analizar los resultados obtenidos de las diferentes combinaciones.
- Extraer conclusiones sobre la pertinencia de combinar técnicas de *Soft Computing* con métodos de XAI, con la intención de comprobar si esta integración puede constituir una línea de trabajo prometedora para mejorar la comprensión de modelos de "caja negra".

Capítulo 3

Estado de la cuestión

Las reglas difusas han sido, tradicionalmente, modelos interpretables. Al igual que la Inteligencia Artificial vivió su particular invierno y ahora ha resurgido como el Ave Fénix, en los últimos años han sido el *Soft Computing* y, concretamente, la Lógica Difusa (*Fuzzy Logic*), los que han sido denostados. Este declive se debe, entre otras razones, a que el Aprendizaje Profundo (*Deep Learning*) lo ha fagocitado todo gracias a la disponibilidad masiva de datos y al aumento exponencial de la potencia y velocidad de cálculo. Sin embargo, este nuevo paradigma predictivo tiene un problema crítico: la falta de explicabilidad. En este Trabajo de Fin de Máster (TFM) se estudia la posibilidad de que las técnicas de *Soft Computing* puedan encontrar su lugar en esta nueva era de la Inteligencia Artificial, aportando soluciones a este problema de la interpretabilidad.

En este capítulo se estructuran los fundamentos sobre los que se asienta la investigación. En la primera mitad, se explicará la problemática actual, introduciendo el marco de la **Inteligencia Artificial Explicable (XAI)** (Sección 3.1) y sus distintas formas. Finalmente, se explicarán en profundidad las herramientas de interpretabilidad que dominan el estado del arte contemporáneo: **SHAP** y **LIME**, los cuales proporcionarán la base operativa sobre la que operará nuestra propuesta metodológica. Posteriormente, en la segunda mitad del capítulo se explorarán las bases teóricas de la **Lógica Difusa** (Sección 3.2), detallando la arquitectura interna de los Sistemas de Inferencia Difusa (Sección 3.2.3) y algunas técnicas interesantes dentro del paradigma del *Soft Computing* (Sección 3.2.4), además de las distintas investigaciones actuales para ambos paradigmas.

3.1. IA Explicable (XAI)

La Inteligencia Artificial Explicable, comúnmente llamada **XAI** por sus siglas en inglés (*eXplainable Artificial Intelligence*), representa uno de los campos de investigación más activos en el panorama de la Inteligencia Artificial actual. A medida que los algoritmos de Aprendizaje Automático (*Machine Learning*) se integran en los procesos de toma de decisiones de la sociedad, surge la gran necesidad de comprender cómo y por qué obtenemos las predicciones generadas por los modelos.

Para establecer un marco teórico riguroso, este trabajo se basa en los conceptos expuestos por Christoph Molnar en su artículo *"Interpretable Machine Learning: A Guide for Making Black Box Models Explainable"* [MOL19]. Molnar define la interpretabilidad como *"el grado en que un ser humano puede comprender de forma consistente la causa de una decisión"*, en otras palabras, *"el grado en que un ser humano puede predecir consistentemente el resultado de un modelo"*. Aunque en la literatura los términos "explicabilidad" e "interpretabilidad" suelen utilizarse de

forma similar, la interpretabilidad se asocia más a las propiedades matemáticas propias de un modelo transparente, mientras que la explicabilidad se refiere a las técnicas aplicadas *a posteriori* para traducir el comportamiento de un modelo opaco a un formato comprensible.

El origen del paradigma XAI: El dilema de la Caja Negra

El surgimiento de la IA Explicable no es un fenómeno aislado, sino la respuesta directa a una limitación estructural del Aprendizaje Automático moderno: el compromiso (*trade-off*) entre el rendimiento predictivo y la transparencia del modelo.

Históricamente, la estadística tradicional y los primeros sistemas de IA se basaban en algoritmos de "caja blanca" o modelos intrínsecamente interpretables, como la regresión lineal, la regresión logística o los árboles de decisión simples. En estos modelos, la relación entre las variables de entrada y la predicción final se basa en coeficientes matemáticos directos o reglas lógicas. Sin embargo, su capacidad para capturar relaciones no lineales y patrones complejos en conjuntos de datos masivos es limitada.

Con el gran avance del *Deep Learning* (Redes Neuronales Profundas) y los métodos de *ensemble* masivo (como *Random Forests* o *Gradient Boosting*), el rendimiento predictivo experimentó un gran salto cualitativo. No obstante, esta precisión se logró a costa de transformar el modelo en una "caja negra". Una red neuronal profunda puede contener miles de millones de parámetros (pesos y sesgos) interactuando simultáneamente mediante funciones de activación no lineales. Tal y como argumenta Molnar [MOL19], la mente humana es incapaz de procesar o comprender mentalmente estas inmensas matrices de números. El modelo sabe *qué* predecir con una precisión asombrosa, pero es incapaz de articular *por qué* ha llegado a esa conclusión.

Beneficios y Necesidad de la Explicabilidad

Cuando un modelo se utiliza para temas de ocio (por ejemplo, para recomendar películas o clasificar imágenes en redes sociales), el coste de una predicción errónea es mínimo, y la opacidad es un precio razonable a pagar por la precisión. Sin embargo, Molnar destaca que la exigencia de explicabilidad se vuelve innegociable cuando los algoritmos operan en dominios de alto riesgo (*high-stakes domains*). Los principales motivos y beneficios que justifican la adopción de técnicas XAI se pueden agrupar de la siguiente manera:

1. **Confianza y Aceptación (*Trust*):** Para que un experto humano delegue una decisión en un algoritmo, primero debe confiar en él. Si un sistema de IA oncológico diagnostica un tumor maligno a partir de una radiografía, el médico especialista no puede (ni debe) aceptar la predicción por fe ciega. La XAI proporciona al profesional la evidencia de las regiones de la imagen que motivaron el diagnóstico, permitiéndole validar si el modelo está utilizando características biológicamente relevantes o si, por el contrario, se está basando en pruebas irrelevantes.

2. **Detección de Sesgos y Equidad (*Fairness*):** Los modelos de *Machine Learning* aprenden de datos históricos generados por humanos y, por esta misma razón, heredan y amplifican sus prejuicios sociológicos. Existen casos documentados donde algoritmos de contratación o de predicción de reincidencia criminal desarrollaron sesgos discriminatorios por género o raza. Las herramientas XAI permiten mostrar qué peso se le está otorgando a estas variables, garantizando que las predicciones sean éticas, justas e imparciales.

3. **Depuración y Mejora del Modelo (*Debugging*):** La explicabilidad es una herramienta de ingeniería fundamental. Permite a los científicos de datos identificar comportamientos falsos. Por ejemplo, en un conocido experimento de clasificación de imágenes entre "lobos"

y "perros husky", la herramienta de explicabilidad reveló que el modelo de caja negra había aprendido a identificar la nieve en el fondo (presente en todas las fotos de lobos) en lugar de las características anatómicas del animal. Sin XAI, este error de generalización habría pasado desapercibido bajo una falsa métrica de alta precisión [MOL19].

4. **Cumplimiento Normativo y Legal:** La interpretabilidad ha pasado más allá del ámbito académico para convertirse en una necesidad legal. Normativas recientes, como el Reglamento General de Protección de Datos (RGPD) en la Unión Europea, introducen el *"derecho a la explicación"*. Esto otorga a los ciudadanos el derecho legal a solicitar información significativa sobre la lógica subyacente de cualquier decisión algorítmica automatizada que les afecte significativamente (por ejemplo, la denegación automática de un crédito bancario).

5. **Descubrimiento Científico:** En disciplinas como la farmacología o la física de partículas, la XAI no solo se usa para evaluar, sino para descubrir nuevo conocimiento. Si un modelo de "caja negra" es capaz de predecir con éxito la eficacia de un compuesto químico, extraer las reglas o características que el modelo ha aprendido puede proporcionar a los investigadores nuevas hipótesis científicas sobre las relaciones funcionales que tiene.

En conclusión, la Inteligencia Artificial Explicable actúa como el puente entre el gran poder computacional de los algoritmos modernos y la necesidad humana de razonamiento ante las predicciones de un modelo. Proveer a estos sistemas de interpretabilidad, ya sea mediante el diseño de modelos transparentes o mediante la aplicación de técnicas agnósticas *post-hoc*, es el paso fundamental para garantizar una Inteligencia Artificial segura, ética y colaborativa.

3.1.1. Tipos de XAI

Dada la rápida expansión de algoritmos y técnicas orientadas a resolver el problema de la caja negra, los investigadores han establecido diversas formas de clasificar las herramientas XAI. Siguiendo el marco conceptual de Christoph Molnar [MOL19], los métodos de interpretabilidad no se evalúan de forma aislada, sino que se categorizan en base a tres características fundamentales: el momento en el que se genera la explicación, la dependencia de la arquitectura del modelo y el alcance o nivel de cobertura de dicha explicación.

A continuación, se explican en profundidad estas tres dimensiones teóricas, las cuales son necesarias para comprender el ecosistema XAI y situar las herramientas que presentamos en nuestro Trabajo de Fin de Máster.

1. Según el momento de la explicación: Intrínsecos vs. *Post-hoc*

Esta característica clasifica los métodos en función de *cuándo* y *cómo* se logra la interpretabilidad dentro del ciclo de vida del aprendizaje automático.

- **Modelos Intrínsecos (Transparentes por diseño):**
  La interpretabilidad intrínseca se consigue disminuyendo la complejidad del modelo de *Machine Learning* desde su creación. De esta manera, el modelo que realiza la predicción y el modelo que proporciona la explicación son exactamente el mismo. Ejemplos clásicos son la Regresión Lineal (donde el peso de los coeficientes explica directamente el impacto de cada variable), la Regresión Logística, los Árboles de Decisión de baja profundidad o los Sistemas de Inferencia Difusa (Mamdani). La ventaja principal es la máxima fidelidad: la explicación es matemáticamente exacta respecto a cómo operó el modelo. La mayor

desventaja es el sacrificio directo del rendimiento predictivo frente a conjuntos de datos altamente no lineales o no estructurados (imágenes, texto, etc.).

- **Métodos *Post-hoc* (Explicación *a posteriori*):**
  En este caso, se asume y se acepta la opacidad del modelo predictivo (por ejemplo, una Red Neuronal Profunda). La explicación no proviene del modelo en sí, sino de una técnica secundaria que se aplica *después* de que el modelo haya sido entrenado y haya generado una predicción. El método *post-hoc* analiza el comportamiento de la caja negra para generar una explicación comprensible de su funcionamiento. Este enfoque permite mantener la máxima precisión predictiva de los algoritmos modernos, a cambio de que la explicación sea una aproximación y no una representación matemática exacta de los pesos internos de la red.

2. Según la dependencia algorítmica: Específicos vs. Agnósticos

Esta característica evalúa el grado de acoplamiento que tiene la técnica de explicabilidad con el algoritmo predictivo.

- **Específicos del Modelo (*Model-Specific*):**
  Son técnicas de interpretabilidad que están totalmente ligadas a una arquitectura concreta de *Machine Learning*. Estos métodos requieren acceso a los componentes internos del modelo (pesos, gradientes, hiperparámetros, etc.). Por ejemplo, el análisis de mapas de activación (Grad-CAM) es un método *post-hoc* pero *model-specific*, ya que solo puede aplicarse sobre Redes Neuronales Convolucionales (CNNs), siendo inútil si el modelo se cambia por un *Random Forest*.

- **Agnósticos al Modelo (*Model-Agnostic*):**
  Según Molnar [MOL19], la interpretabilidad agnóstica es el objetivo más ambicioso de la ciencia de datos moderna. Estos métodos operan bajo la premisa estricta de que el modelo es una caja negra. No requieren conocer los gradientes ni los pesos internos. Su funcionamiento se basa exclusivamente en la perturbación de los datos de entrada y la observación de cómo dichas variaciones alteran las predicciones de salida. La inmensa ventaja teórica de los métodos agnósticos es su modularidad: permiten sustituir un algoritmo predictivo obsoleto por una arquitectura más novedosa y precisa sin tener que eliminar ni reescribir la herramienta de explicabilidad.

3. Según el alcance de la explicación: Local vs. Global

Esta característica determina si la técnica de explicabilidad intenta resumir el comportamiento completo del sistema o si se centra en explicar una decisión concreta e individual.

- **Explicabilidad Global:**
  Un método global busca proporcionar al usuario una comprensión completa de cómo el modelo toma decisiones sobre todo el espacio de características. Responde a preguntas de alto nivel, como: *"En general, ¿qué variables son las más determinantes para que el modelo conceda una hipoteca?"*. Aunque es fundamental para comprobar sesgos a nivel general, la explicabilidad global es normalmente difícil (o casi incomprensible para una persona) cuando el modelo de caja negra tiene cientos de variables que interactúan de forma no lineal.

- **Explicabilidad Local:**
  En contraste, la explicabilidad local aísla una única instancia (por ejemplo, un único paciente o un único cliente de un banco) y responde a la pregunta de por qué el modelo ha

tomado una decisión específica para *un caso particular*. Es posible que, a nivel global, los "ingresos" sean la variable más importante del modelo, pero que en la evaluación local de un cliente concreto, la razón principal del rechazo del préstamo haya sido un "impago reciente" o la "puntuación crediticia" (*credit score*). Molnar argumenta que las explicaciones locales suelen ser mucho más precisas y útiles para el usuario final.

El análisis de esta clasificación muestra que no existe una única solución XAI perfecta, sino que existen herramientas diseñadas para diferentes necesidades de explicabilidad. Si se decide usar modelos **intrínsecos**, se corre el riesgo de no poder procesar datos masivos con la precisión requerida actualmente. Si se opta por métodos **específicos del modelo**, se genera una dependencia, atando el ecosistema de explicabilidad a una única arquitectura. Y si se intenta forzar una explicación **global** sobre un modelo complejo de cientos de variables, el resultado suele ser incomprensible para la persona especializada que necesita las explicaciones.

Es precisamente por estas limitaciones por lo que se ha realizado un consenso en la explicabilidad: para explicar con éxito modelos complejos en entornos de producción, el paradigma más robusto es la intersección entre los métodos ***post-hoc*, agnósticos al modelo y de alcance local**.

Este cruce permite al experto emplear cualquier modelo de caja negra (*post-hoc* y agnóstico), y obtener una explicación detallada, precisa y matemáticamente abordable para cada decisión individual (local).

Las dos herramientas que dominan en la actualidad son **LIME** y **SHAP**. Al trabajar como explicadores locales y agnósticos, ambos métodos logran aproximar problemas altamente no lineales mediante modelos subrogados más sencillos. A partir de este marco teórico, los siguientes subapartados (Sección 3.1.2 y 3.1.3) se dedicarán a explicar el funcionamiento tanto matemático y como algorítmico de estas dos herramientas.

3.1.2. Shap

SHAP, siglas de *SHapley Additive exPlanations*, es uno de los marcos metodológicos más robustos en la actualidad dentro de la Inteligencia Artificial Explicable. Presentado en 2017 por Scott M. Lundberg y Su-In Lee [LUN17], SHAP no nació como una técnica aislada, sino como un intento de unificación. Los autores demostraron que varios métodos de interpretabilidad existentes en aquel momento (incluyendo LIME y DeepLIFT) pertenecían a una misma clase teórica llamada "Métodos de Atribución de Características Aditivas". SHAP se entiende como la solución matemática óptima dentro de esta clase.

Para comprender en profundidad cómo funciona SHAP, es necesario separar su implementación algorítmica de sus conceptos teóricos, los cuales se remontan a la Teoría de Juegos de la década de los 50.

Fundamentos Teóricos: Teoría de Juegos Cooperativos

El núcleo matemático de SHAP se basa en los **Valores Shapley** (*Shapley Values*), un concepto introducido por el premio Nobel de Economía Lloyd Shapley en 1953. El problema original de Shapley consistía en determinar cómo distribuir de forma justa el beneficio obtenido por una coalición de jugadores, asumiendo que cada jugador contribuye de manera desigual al éxito colectivo [JOS25].

En el contexto del *Machine Learning*, el enfoque de la XAI traduce este problema económico a uno predictivo mediante la siguiente analogía [JOS25]:

- El **"Juego"** es la tarea de predecir una instancia individual en el conjunto de datos.
- Los **"Jugadores"** son las características o variables de entrada (*features*) de esa instancia concreta.
- La **"Ganancia" o "Pago"** es la diferencia entre la predicción real para esa instancia y la predicción media global de todo el modelo.

El objetivo de los valores Shapley es repartir esta "ganancia" (la variación en la predicción) entre todos los "jugadores" (las variables) de forma equitativa, calculando la contribución marginal de cada variable a medida que se une a la coalición [JOS25].

Formulación Matemática de los Valores Shapley

Para formalizar este cálculo, la contribución exacta (el valor Shapley) de una característica i, definida como ϕ_i, se define mediante la siguiente ecuación [JOS25]:

$$\phi_i = \sum_{s \subseteq \{x_1,\ldots,x_p\}\{x_i\}} \frac{|s|!(p-|s|-1)!}{p!}(value(s \cup \{x_i\}) - value(s)) \tag{3.1}$$

Donde:

- p es el número total de características.
- $p!$ es el número total de combinaciones entre características.
- s es un subconjunto de características (coalición) que no contiene a la característica (jugador) x_i.
- $s!$ es el número de formas en las que se puede formar una coalición s.
- $(p-|s|-1)!$ es el número de maneras en las que un jugador (característica) se puede unir a una coalición s después de que se haya unido a dicha coalición el jugador (característica) x_i.
- $value(s)$ es la predicción del modelo utilizando únicamente las características del subconjunto s.
- $value(s \cup \{x_i\}) - value(s)$ representa la **contribución marginal** de la característica i cuando se añade al subconjunto s.
- La fracción $\frac{|s|!(p-|s|-1)!}{p!}$ es el peso de ese subconjunto (coalición). Representa la proporción de permutaciones en las que la coalición s se forma antes de que se añada la característica i.

Esta fórmula garantiza que el valor de impacto asignado a cada característica considera todas las posibles interacciones con el resto de variables del sistema.

De Shapley a SHAP: El Marco de Atribución Aditiva

Aunque los valores Shapley proveen una base teórica sólida, calcularlos de forma exacta para un modelo de *Machine Learning* requiere 2^p evaluaciones, lo cual es computacionalmente inviable para cualquier conjunto de datos moderno con decenas o cientos de variables [JOS25].

La solución propuesta por Lundberg y Lee [LUN17] para este problema fue crear **SHAP**, una solución que aproxima estos valores de forma eficiente para modelos de IA. SHAP define las explicaciones como un modelo subrogado de atribución aditiva de características:

$$g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i z'_i \tag{3.2}$$

Donde g es el modelo explicativo, $z'_i \in \{0, 1\}$ es una variable binaria que indica si una característica está presente o ausente, M es el número de características de entrada simplificadas, y ϕ_i son los valores SHAP calculados.

El éxito de SHAP se debe a que Lundberg y Lee demostraron teóricamente que es el **único** método de explicabilidad aditiva que satisface simultáneamente tres propiedades matemáticas deseables [LUN17]:

1. **Precisión Local (*Local Accuracy* o Eficiencia):** La suma de los valores SHAP de todas las características debe ser igual a la salida real del modelo para esa instancia menos la salida esperada del modelo base (ϕ_0).

2. **Carencia o Ausencia (*Missingness*):** Si una característica está ausente en la instancia original ($z'_i = 0$), su contribución o valor SHAP asignado debe ser obligatoriamente cero.

3. **Consistencia (*Consistency*):** Si se modifica la arquitectura o los pesos del modelo de caja negra de forma que una característica i tenga un impacto mayor o igual en la predicción final, su valor SHAP resultante nunca debe disminuir. Esta propiedad es vital para garantizar que el experto encargado de analizar los resultados pueda confiar ciegamente en la explicación.

Beneficios y Métodos de Aproximación en la Práctica

La combinación de las bases matemáticas de la teoría de juegos con el aprendizaje automático proporciona a SHAP una forma de explicación única y muy sólida [JOS25]:

- **Interpretabilidad Local:** Muestra un desglose exacto (positivo o negativo) de cómo cada variable ha contribuido a la predicción de una instancia específica por encima o por debajo del valor base esperado.

- **Interpretabilidad Global:** Al agregar y promediar los valores SHAP absolutos de todas las instancias del conjunto de entrenamiento, se obtiene una matriz de importancia de características global (*Feature Importance*) superior a las tradicionales.

Para evitar el problema de la explosión combinatoria computacional, SHAP cuenta con optimizaciones específicas. La más destacada es **KernelSHAP** [LUN17], un método agnóstico al modelo que utiliza una regresión lineal ponderada inspirada fuertemente en la lógica de vecinos de LIME, pero con pesos, datos sintéticos y penalizaciones matemáticamente ajustadas para que la regresión converja exactamente hacia los valores de Shapley.

En resumen, SHAP destaca como una de las herramientas más precisas desde el punto de vista matemático dentro de la la IA Explicable. Sin embargo, sus resultados siguen siendo principalmente numéricos: ofrece vectores numéricos de alta precisión, pero no aporta una interpretación en lenguaje más cercano o intuitivo, como sí podría hacerlo un enfoque basado en lógica difusa. Por eso, en esta investigación se plantea explorar métodos alternativos para solucionar esta limitación.

3.1.3. LIME

Dentro del tipo de métodos *post-hoc* y agnósticos al modelo, **LIME** (*Local Interpretable Model-agnostic Explanations*) se ha consolidado como uno de los enfoques más básicos y fundamentales. Propuesto en 2016 por Marco Tulio Ribeiro, Sameer Singh y Carlos Guestrin en su artículo *"Why should I trust you?": Explaining the predictions of any classifier* [RIB16], LIME ofrece una solución elegante al problema de la "caja negra" aproximando modelos complejos mediante modelos simples en vecindarios locales.

Confianza (*Trust*): Predicción vs. Modelo

El desarrollo de LIME parte de una base social: si los usuarios no confían en un modelo o en una predicción, no lo utilizarán, independientemente de cuán altas sean sus métricas de precisión teóricas (como el *Accuracy* o el *F1-Score*). Ribeiro et al. [RIB16] exponen explícitamente dos niveles de confianza que las herramientas de IA Explicable deben satisfacer:

1. **Confianza en la predicción (*Trusting a prediction*):** Es la necesidad de un usuario final (por ejemplo, un médico) de entender por qué el modelo ha tomado una decisión específica para una instancia concreta (por ejemplo, un paciente) antes de tomar una acción basada en dicha predicción.

2. **Confianza en el modelo (*Trusting a model*):** Es la necesidad del ingeniero o científico de datos de evaluar si el modelo, en su conjunto, es robusto y se comportará de manera razonable cuando se despliegue en un entorno real, asegurando que no está explotando sesgos del conjunto de entrenamiento.

LIME clásico nace para resolver el primer nivel de confianza mediante explicaciones locales. Su idea principal es que, aunque las decisiones de una caja negra sean indescifrables y altamente no lineales, cualquier decisión compleja puede aproximarse de forma lineal si se examina desde un punto de vista lo suficientemente cercano (un vecindario local) a la instancia a evaluar.

Representación de Datos Interpretable

Una de las grandes aportaciones de LIME es la distinción conceptual entre las características que utiliza el modelo de aprendizaje automático y las características que comprende la persona que revisa la salida de dicho modelo [RIB16].

Sea $x \in \mathbb{R}^d$ el vector original de características que se le proporciona a la caja negra. En modelos modernos, estas características pueden ser abstractas y puede carecer de significado directo para una persona (por ejemplo, los *embeddings* de palabras en Procesamiento de Lenguaje Natural, o los valores RGB individuales de los píxeles en una imagen).

Para generar una explicación, LIME define una **representación interpretable** $x' \in \{0,1\}^{d'}$. Se trata de un vector binario que indica la "presencia" o "ausencia" de componentes semánticos

comprensibles. En el caso de un texto, x' podría representar la presencia/ausencia de palabras concretas (una bolsa de palabras o *bag of words*); en imágenes, la presencia/ausencia de superpíxeles contiguos; y en datos tabulares, la pertenencia a ciertos rangos de valores.

LIME desde un punto de vista teórico

El objetivo de LIME es encontrar un modelo explicativo g (por ejemplo, una regresión lineal o un árbol de decisión) perteneciente a una clase de modelos interpretables G, que aproxime el comportamiento del modelo de "caja negra" f en el entorno local de la instancia x.

Para garantizar que la explicación sea útil, LIME define la siguiente función objetivo a minimizar [RIB16]:

$$\xi(x) = \text{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{3.3}$$

Esta ecuación es la base del marco teórico de LIME. Cada término cumple una función restrictiva importante:

- $\mathbf{f}(\mathbf{x})$: Es la probabilidad (o valor de regresión) que emite el modelo de caja negra original.
- $\mathbf{g} \in \mathbf{G}$: Es el modelo subrogado interpretable. En el LIME clásico, G suele ser la clase de los modelos lineales (ej. *Ridge Regression* o *Lasso*).
- $\pi_{\mathbf{x}}(\mathbf{z})$: Es la medida de proximidad o vecindad. Define cómo de grande es la distancia desde z a la instancia original x que estamos intentando explicar. LIME suele emplear una función de distancia exponencial (*Kernel*), donde las instancias z más cercanas a x tienen un peso mucho mayor que las lejanas.
- $\mathcal{L}(\mathbf{f}, \mathbf{g}, \pi_{\mathbf{x}})$: Es la función de pérdida de fidelidad local. Mide cómo de infiel (*unfaithful*) es el modelo interpretable g a la hora de imitar las predicciones del modelo de "caja negra" f en el vecindario definido por π_x.
- $\mathbf{\Omega}(\mathbf{g})$: Es la medida de penalización por complejidad. Garantiza que la explicación resultante sea asimilable por un humano. Si g es un árbol de decisión, $\Omega(g)$ podría ser la profundidad del árbol. Si g es un modelo lineal, $\Omega(g)$ es el número de coeficientes distintos de cero.

LIME resuelve este problema de optimización intentando minimizar la pérdida de fidelidad local $\mathcal{L}$ mientras mantiene la complejidad Ω lo suficientemente baja para ser interpretable [RIB16].

LIME en la práctica

Para ejecutar la ecuación anterior sin conocer las matemáticas internas de f (naturaleza agnóstica), LIME ejecuta el siguiente algoritmo:

1. **Perturbación de la instancia:** A partir de la representación interpretable x', LIME genera un vecindario de datos sintéticos o perturbados (Z'). Esto se logra activando o desactivando aleatoriamente (modificando entre 1 y 0) características de x'.
2. **Traducción al espacio original:** Cada instancia sintética z' se traduce de vuelta a su representación original $z \in \mathbb{R}^d$ en el espacio que el modelo de caja negra puede entender.

3. **Inferencia del modelo de "caja negra":** Se introducen todas las instancias perturbadas z en el modelo opaco f para obtener sus respectivas predicciones $f(z)$. Este paso mapea cómo responde el modelo ante variaciones en la entrada.

4. **Ponderación por distancia:** Cada instancia sintética se pondera mediante el *kernel* exponencial π_x. Las perturbaciones que sean muy similares a la instancia original x tendrán un peso decisivo en la explicación, mientras que las variaciones extremas tendrán un impacto casi nulo.

5. **Ajuste del modelo subrogado:** Utilizando el conjunto de datos perturbados, ponderados y etiquetados por la "caja negra", LIME entrena un modelo interpretable g (típicamente una regresión lineal). Los pesos finales de esta regresión lineal conforman la explicación que se le presenta al usuario final.

Beneficios y Limitaciones

El diseño propuesto por Ribeiro et al. [RIB16] aporta ciertos beneficios a la ciencia de datos. Al ser agnóstico, LIME permite crear explicaciones para Redes Neuronales, *Support Vector Machines* (SVMs) o *Random Forests* utilizando exactamente el mismo *pipeline* de ejecución. Además, al basarse en una representación interpretable x', las explicaciones no se expresan en el lenguaje matemático del algoritmo, sino en un formato más cercano y comprensible para la persona experta.

Sin embargo, en su intento por proporcionar explicaciones simples, el LIME tradicional presenta una limitación significativa: reduce la explicación de cualquier entorno a una simple sumatoria de valores reales que expresan importancia. Como se planteó en apartados anteriores, las personas a menudo se basan en reglas lógicas y difusas para entender las explicaciones de un problema, y no en coeficientes matemáticos estáticos.

Por otro lado, aunque LIME resuelve el problema de la confianza local (confiar en la predicción), explicar una instancia aislada no es suficiente para evaluar las decisiones de un modelo opaco en su totalidad. Para comprender el sistema completo y lograr la confianza en el modelo, es necesario seleccionar un subconjunto representativo de instancias, dando origen a la técnica conocida como **SP-LIME** (*Submodular Pick LIME*), la cual se explica a continuación.

Submodular Pick LIME

El algoritmo base de LIME resuelve de manera efectiva el problema de la confianza en la predicción, permitiendo a un usuario evaluar una decisión individual. Sin embargo, cuando el objetivo de un ingeniero de datos es comprobar si un modelo opaco está listo para ser usado, evaluar una única instancia es totalmente insuficiente. Para conseguir la "confianza en el modelo" (*trusting a model*), es necesario entender el comportamiento global del modelo de caja negra.

Para su experimento, Ribeiro, Singh y Guestrin [RIB16] asumieron que las personas tienen un límite (tiempo o paciencia) sobre el número de explicaciones a analizar (dicho limite lo denominaron B, de presupuesto o *budget*). Un experto no puede permitirse el lujo de leer y analizar las explicaciones locales de miles de instancias. Por tanto, el análisis consiste en seleccionar un subconjunto de B instancias concretas (por ejemplo, $B = 10$ o 20) cuyas explicaciones locales, en conjunto, proporcionen la visión más completa y representativa posible de cómo razona el modelo globalmente.

Aproximar esta visión global seleccionando instancias al azar es una estrategia bastante ineficiente. El muestreo aleatorio suele generar redundancia: si un modelo de clasificación de correos electrónicos clasifica los mensajes de "spam" basándose en la palabra *"gratis"*, elegir 10 correos al azar probablemente devolverá 10 explicaciones idénticas que se justifican mediante la palabra *"gratis"*, desaprovechando el presupuesto B y ocultando otras variables menos frecuentes pero igualmente importantes.

Para resolver esta ineficiencia, los autores proponen **SP-LIME** (*Submodular Pick LIME*), un algoritmo elegante que transforma el problema de la selección de instancias en un problema de optimización de cobertura de características.

Formulación Matemática de la Cobertura

El funcionamiento de SP-LIME se basa en la construcción de una matriz espacial de explicaciones y la maximización de una función submodular [RIB16]. El proceso se realiza siguiendo los siguientes pasos:

1. **Construcción de la Matriz de Explicaciones (W):** En primer lugar, se ejecuta el algoritmo LIME estándar sobre un conjunto grande de instancias X (por ejemplo, todo el conjunto de entrenamiento). Se construye una matriz W de dimensiones $n \times d'$, donde n es el número de instancias y d' es el número total de características interpretables disponibles. Una celda W_{ij} contiene la magnitud del peso local (la importancia) que la característica j tiene en la explicación de la instancia i. Si una característica no aparece en la explicación de una instancia, su valor es cero.

2. **Cálculo de la Importancia Global (I):** Para evaluar qué características son las más relevantes a nivel general, se calcula un vector de importancia global I. Una forma estándar de calcular la importancia global I_j de la característica j es sumar la magnitud de sus pesos locales a lo largo de todas las instancias evaluadas:

$$I_j = \sqrt{\sum_{i=1}^{n} |W_{ij}|}$$

3. **Definición de la Función de Cobertura (c):** Dado un conjunto V de instancias seleccionadas (donde $|V| \leq B$), la función de cobertura mide cuánta de la importancia global del modelo logramos explicar o mostrar al usuario si solo le enseñamos las explicaciones del conjunto V. Matemáticamente, la cobertura se define evaluando qué características están activas en el subconjunto V [RIB16]:

$$c(V, W, I) = \sum_{j=1}^{d'} \mathbb{1}_{[\exists i \in V : W_{ij} > 0]} \cdot I_j$$

   La función indicadora $\mathbb{1}$ toma el valor 1 si la característica j aparece con un peso mayor que cero en al menos una de las explicaciones del conjunto V. Lo fundamental de esta ecuación es que evita la redundancia: una vez que una característica ya ha sido mostrada en una de las explicaciones del conjunto V, no suma más cobertura aunque aparezca en las demás. Esto obliga al método a buscar diversidad.

Resolución del Problema de Optimización (Algoritmo *Greedy*)

El objetivo final de SP-LIME es encontrar el conjunto V^* que maximice la función de cobertura c, respetando el límite B:

$$V^* = \text{argmax}_{V,|V|\leq B}\ c(V, W, I)$$

Desde el punto de vista algorítmico, este es un problema clásico de cobertura máxima ponderada, el cual no se puede resolver de forma exacta en un tiempo computacional razonable para conjuntos de datos grandes (es *NP-hard*) [RIB16].

Sin embargo, debido a que la función de cobertura c tiene la propiedad de ser *submodular*, los autores aplican un algoritmo codicioso (*greedy*) para aproximar la solución. El algoritmo *greedy* se ejecuta de forma iterativa: en cada paso, evalúa todas las instancias no seleccionadas y añade al conjunto V aquella instancia que aporte el mayor aumento a la cobertura total, hasta alcanzar el límite B. Aunque es una aproximación, teóricamente se garantiza que este algoritmo iterativo logrará al menos un 63 % (exactamente $1 - 1/e$) del valor óptimo absoluto.

Beneficios y Relevancia Práctica

La implementación de SP-LIME pone a la disposición de los expertos de *Machine Learning* una herramienta de explicabilidad global realmente potente. Sus beneficios principales son los siguientes:

- **Diversidad Semántica:**
  Al penalizar la redundancia, garantiza que el panel de explicaciones (*dashboard*) mostrado abarque la mayor cantidad de características relevantes posible del comportamiento del modelo.

- **Explicaciones de Casos Límite:**
  Facilita la detección de comportamientos anómalos o atajos (*shortcuts*) aprendidos por el modelo que, mediante un simple muestreo aleatorio, habrían pasado desapercibidos bajo la sombra de características más predominantes.

- **Economía del conocimiento:**
  Maximiza la cantidad de información extraída por el usuario minimizando el tiempo de lectura (presupuesto B).

En resumen, SP-LIME demuestra que la interpretabilidad local no tiene por qué estar aislada del entendimiento general. Seleccionando estratégicamente un subconjunto de explicaciones locales independientes, es posible construir una explicación global suficientemente precisa del modelo de "caja negra".

Aunque la arquitectura de SP-LIME resuelve perfectamente el problema de la explicabilidad global, sus explicaciones finales siguen limitadas por la rigidez de los coeficientes lineales. Para superar esta barrera necesitamos integrar herramientas que simulen de forma más natural el lenguaje humano. Este desafío es una de las justificaciones de la propuesta de este Trabajo de Fin de Máster: fusionar las perturbaciones de LIME con la semántica de la lógica difusa.

3.1.4. Estado actual de la IA Explicable

En este apartado, hablaremos de la situación actual de la Inteligencia Artificial Explicable. Para ello exploraremos qué soluciones existen hoy en día para intentar entender los modelos de "caja negra". En concreto, nos centraremos en repasar las variantes más representativas de las dos herramientas más conocidas de la Inteligencia Artificial Explicable: LIME y SHAP.

El objetivo fundamental de la XAI no es sustituir a los modelos complejos por otros más simples, sino desarrollar técnicas e interfaces capaces de interpretar el comportamiento de estas

cajas negras *a posteriori*. Dentro de este paradigma, los métodos *agnósticos al modelo* han acaparado las miradas de la comunidad científica en los últimos años.

Entre las múltiples aproximaciones existentes, como vimos anteriormente, dos herramientas se han consolidado como los pilares para la generación de explicaciones locales: **LIME** (*Local Interpretable Model-agnostic Explanations*) y **SHAP** (*SHapley Additive exPlanations*).

Aunque los fundamentos de ambas herramientas se detallaron anteriormente, esta sección tiene como propósito explorar sus estados actuales. En primer lugar, se analizará la evolución de LIME y las distintas variantes que han surgido de dicha herramienta. Posteriormente, se examinará el entorno de SHAP, prestando especial atención a una de las modificaciones más recientes y relevantes de la literatura para nuestro trabajo: la integración de valores SHAP con explicaciones lingüísticas difusas, un avance que supone una motivación directa y el punto de partida conceptual para el desarrollo de las herramientas propuestas en este Trabajo de Fin de Máster.

Variantes de LIME

Desde su presentación por Ribeiro et al. en el año 2016 [RIB16], LIME se ha establecido como uno de los pilares fundamentales dentro de las técnicas de interpretabilidad *post-hoc*. Su premisa conceptual, la cual se basa en aproximar el comportamiento de un modelo opaco mediante un modelo subrogado más sencillo y comprensible en un entorno local restringido, supuso un cambio de paradigma en la forma de examinar los algoritmos de aprendizaje automático.

La enorme popularidad de LIME, sumada a las limitaciones inherentes de su diseño original (especialmente en la estabilidad de las explicaciones y el muestreo de datos), ha provocado que la herramienta original evolucione hasta convertirse en un ecosistema en constante expansión. Durante los últimos años, la literatura académica ha documentado numerosas adaptaciones diseñadas para optimizar su rendimiento frente a naturalezas de datos específicas o para dotar al algoritmo de mayor robustez. A continuación, se examinan las variantes más relevantes de esta herramienta.

Desde la concepción original de LIME, la comunidad investigadora ha detectado diversas áreas de mejora, principalmente relacionadas con la inestabilidad de las explicaciones generadas (donde dos ejecuciones sobre la misma instancia pueden arrojar resultados distintos) y la falta de realismo en los datos perturbados. Para mitigar estas deficiencias, han surgido múltiples variantes metodológicas. A continuación, se analizan las extensiones más destacadas en el estado del arte reciente obtenidas de la web *"Which LIME should I trust?"* [PAT25], con especial énfasis en aquellas que aplican sobre datos tabulares o proponen paradigmas novedosos de visualización y muestreo.

gLIME (2021): Metodología Gráfica de Explicaciones

Una de las limitaciones cognitivas del LIME clásico es que proporciona la importancia de las características de forma aislada y unidimensional (un simple vector de valores reales que expresan importancia). Este enfoque asume implícitamente que las variables de entrada son independientes entre sí en su contribución a la predicción, ignorando las interacciones complejas. Para solucionar este problema, Dikopoulou et al. propusieron en 2021 **gLIME** [ZOU21], una novedosa metodología gráfica para explicaciones agnósticas al modelo.

La innovación principal de gLIME reside en la fusión del algoritmo base de LIME con el operador GLASSO (*Graphical Least Absolute Shrinkage and Selection Operator*). En lugar de

generar una simple ecuación lineal subrogada, gLIME construye Modelos Gráficos Gaussianos no dirigidos. Este enfoque permite no solo identificar qué características son críticas para la red neuronal, sino también revelar las relaciones condicionales entre ellas. El resultado es un grafo interpretable donde los nodos representan las características y las aristas cuantifican la fuerza de las relaciones, capturando tanto el impacto directo como el indirecto sobre la decisión del modelo.

Otra ventaja de gLIME es que este enfoque está diseñado para proporcionar explicaciones a escala local (para una instancia específica) y a escala global (agrupando el comportamiento sobre el conjunto de datos completo). En sus experimentos sobre conjuntos de datos médicos reales (como BIOPSY), los autores demostraron que gLIME no solo mejora la comprensión humana de las relaciones entre las variables, sino que logra un incremento notable en la consistencia de las explicaciones frente a permutaciones, elevando la estabilidad de la importancia de las variables del $52\,\% - 77\,\%$ (que consigue el LIME clásico) hasta un $76\,\% - 96\,\%$ (conseguido por gLIME).

GLIME (2023): Estabilidad y Muestreo Generalizado

Aunque a menudo se confunde en la literatura por sus siglas similares, **GLIME** (*General, Stable and Local LIME Explanation*), propuesto por Tan et al. en la conferencia NeurIPS en 2023 [ZER23], cubre un problema distinto: la inestabilidad matemática del muestreo estocástico. El algoritmo clásico de LIME sufre de una fidelidad local subóptima debido a que su espacio de muestreo genera datos sintéticos que a menudo se alejan demasiado del vecindario real de la instancia a explicar.

El marco de trabajo de GLIME aborda este problema demostrando teóricamente que el cuello de botella de LIME radica en separar la generación de perturbaciones de la ponderación de las distancias. Para solucionarlo, los autores proponen una formulación matemática equivalente que integra la función de ponderación directamente en la distribución de muestreo. Esto da lugar a una instancia especializada denominada *GLIME-BINOMIAL*, la cual utiliza una distribución local y no sesgada para generar el vecindario.

Desde un punto de vista algorítmico, esta modificación garantiza que los puntos sintéticos generados orbiten estrictamente alrededor de la instancia de interés con una relevancia matemática garantizada. Los resultados demostraron que GLIME-BINOMIAL logra una aceleración exponencial en la convergencia del regresor lineal subrogado. Al requerir muchas menos muestras para estabilizarse, la herramienta no solo disminuye drásticamente el coste computacional del explicador, sino que elimina el comportamiento engañoso del LIME original donde diferentes semillas aleatorias (*random seeds*) alteraban los pesos finales de la explicación.

ExpLIMEable (2023): Análisis Visual y Exploratorio

A diferencia de las variantes anteriores que modifican el núcleo matemático de LIME, **ExpLIMEable** [SON23] se presenta como un marco de análisis visual integral centrado en el enfoque de *Human-in-the-Loop*. Este enfoque ha sido desarrollado principalmente para evaluar redes neuronales convolucionales en el ámbito de la detección de imágenes médicas, sin embargo, su filosofía se puede extender a la ciencia de datos en general.

El problema que motiva ExpLIMEable es la gran sensibilidad de LIME a sus hiperparámetros. En entornos como la medicina o las finanzas, una única explicación local estática no genera la suficiente confianza, ya que la variación del tamaño del vecindario (*kernel width*) o del método de perturbación puede alterar bastante la interpretación generada. ExpLIMEable soluciona esto mostrando un panel de control visual (*dashboard*) que permite a los expertos en *Machine*

Learning explorar el espacio de explicaciones en tiempo real.

Una de las aportaciones más relevantes de este marco es la introducción de una fase de reducción de dimensionalidad sobre las perturbaciones locales generadas por LIME. Al proyectar estos datos sintéticos en un espacio visual, el analista puede observar gráficamente la topología del vecindario y juzgar si las muestras son informativas o si es necesario modificar los parámetros. Además, ExpLIMEable implementa un sistema de rastreo de procedencia (*provenance tracking*), permitiendo al usuario navegar y registrar cómo evoluciona la explicación a medida que se ajustan las suposiciones del modelo subrogado, elevando así la robustez y confiabilidad general del proceso.

Esta variante es de gran interés para nuestro Trabajo de Fin de Máster, ya que presenta una idea interesante: el éxito de la Inteligencia Artificial Explicable no solo depende del rigor matemático del modelo subrogado (como nuestras reglas difusas), sino de la capacidad de envolverlo en una Interfaz de Usuario (API) que facilite su comprensión a los usuarios finales.

DLIME (2019): LIME Determinista para Datos Tabulares

En el contexto específico de los conjuntos de datos tabulares, la debilidad más documentada de LIME es la generación de perturbaciones fuera de la distribución (*Out-of-Distribution*). Al alterar características continuas mediante ruido gaussiano sin considerar la correlación real entre las columnas, LIME clásico puede generar instancias sintéticas matemáticamente imposibles, lo que contamina el modelo lineal local.

Para solucionar este problema y asegurar una replicabilidad total, Zafar y Khan propusieron en 2019 su herramienta **DLIME** (*Deterministic Local Interpretable Model-Agnostic Explanations*) [MUH19]. DLIME elimina por completo la generación estocástica de datos. En su lugar, utiliza el conjunto de datos de entrenamiento real para construir el vecindario de la instancia a explicar.

El algoritmo opera en dos fases. En primer lugar, aplica un algoritmo de Agrupamiento Jerárquico Aglomerativo (*Agglomerative Hierarchical Clustering*) sobre todo el conjunto de entrenamiento para organizar los datos en clústeres densos y coherentes. Cuando se solicita la explicación de una nueva predicción, DLIME identifica a qué clúster pertenece la instancia y, posteriormente, aplica el algoritmo de K-Vecinos Más Cercanos (K-NN) para seleccionar las instancias reales más próximas dentro de dicho grupo. Este subconjunto real actúa como el vecindario local sobre el que se entrena el regresor lineal. Al no generar datos sintéticos, DLIME garantiza que las características respetan las distribuciones estadísticas originales y proporciona explicaciones totalmente estables, resolviendo el problema del determinismo en entornos tabulares.

LIME Difuso

Como podemos comprobar, en las implementaciones que encontramos de LIME, no existe ninguna que utilice reglas o lógica difusa para ayudar a las explicaciones. Por esta razón hemos encontrado una motivación en crear una versión de LIME que lo implemente.

Como se ha podido comprobar a lo largo de esta revisión, el ecosistema de LIME es bastante variado. La comunidad científica ha propuesto soluciones distintas para mejorar la estabilidad matemática de la herramienta, corregir los problemas de muestreo en datos tabulares e incluso explorar nuevas formas de visualización gráfica. Sin embargo, resulta llamativo que, dentro de estas modificaciones, no exista ninguna variante que utilice de alguna forma la Lógica Difusa o

la extracción de reglas lingüísticas para realizar sus explicaciones.

Todas las implementaciones actuales de LIME continúan ancladas en la devolución de coeficientes matemáticos o, en el mejor de los casos, en la presentación visual de dichos números. Esta falta de semántica humana en las explicaciones de LIME representa una buena oportunidad para la investigación. Precisamente, este vacío en el estado del arte ha sido una de las motivaciones principales de nuestro Trabajo de Fin de Máster: diseñar y desarrollar la primera adaptación de LIME capaz de expresarse mediante conjuntos difusos, acercando la herramienta a la forma natural en la que piensan las personas.

SHAP

Junto a LIME, el otro gran estándar dentro de la Inteligencia Artificial Explicable agnóstica al modelo es **SHAP** [LUN17]. Presentado por Lundberg y Lee en 2017, este marco de trabajo aborda el problema de la interpretabilidad desde una perspectiva distinta: la Teoría de Juegos Cooperativos. Como vimos anteriormente, en lugar de ajustar modelos lineales subrogados en vecindarios perturbados, SHAP evalúa la contribución marginal de cada característica al resultado final de la predicción, calculando lo que se conoce como valores *Shapley* (*Shapley Values*).

Sin embargo, SHAP hereda una limitación cognitiva similar a la de LIME: su salida final sigue siendo un vector numérico de atribuciones de características. Para un experto en ciencia de datos, visualizar que la variable x_1 tiene un valor SHAP de +2,4 puede ser informativo, pero para el usuario final en la toma de decisiones (como un médico por ejemplo), este nivel de abstracción matemática sigue suponiendo una barrera de entrada considerable.

Por ese motivo, se han comenzado a explorar nuevas vías para aportar explicaciones, destacando la integración de valores SHAP con técnicas de *Soft Computing*.

Fuzzy Linguistic Explanations from SHAP Values

Recientemente, la comunidad científica ha comenzado a abordar la falta de interpretabilidad semántica de los vectores SHAP mediante la integración de técnicas de *Soft Computing*. Un avance relevante en esta línea es la herramienta **SHAP-Rule**, presentada por Matrenin et al. (2025) en la revista *Mathematics* (MDPI) bajo el título *"Interpretable Diagnostics with SHAP-Rule: Fuzzy Linguistic Explanations from SHAP Values"* [PAV25].

Esta herramienta propone una innovación sobre la salida tradicional de SHAP para tareas de diagnóstico. En lugar de limitarse a mostrar al usuario un vector numérico con las contribuciones aisladas de cada característica (los valores Shapley), SHAP-Rule implementa un *pipeline* computacional que convierte estas atribuciones cuantitativas en **reglas lingüísticas difusas del tipo SI-ENTONCES (*IF-THEN*)**.

Para lograr esto, el método proyecta los valores de impacto marginal calculados por SHAP sobre un conjunto de funciones de pertenencia aplicando la lógica difusa. De este modo, los umbrales numéricos rígidos se traducen en representaciones simbólicas comprensibles por humanos. Además, la herramienta no solo genera el texto de la regla, sino que cada explicación muestra la fuerza de activación de sus reglas. En sus experimentos sobre conjuntos de datos médicos e industriales, los autores demostraron que esta traducción difusa mejoraba la comprensión de los analistas, manteniendo al mismo tiempo una fidelidad global que se aproximaba al 100 %.

Motivación para el desarrollo de nuestro trabajo

El éxito de SHAP-Rule demuestra que la hibridación de marcos de explicabilidad matemática con lógica difusa es una vía posible para humanizar la comprensión de modelos opacos, superando la barrera cognitiva que suponen las salidas numéricas.

Al observar que metodologías como SHAP han logrado integrar estas reglas lingüísticas para mejorar su interpretabilidad, nos surge la motivación de hacer algo similar con LIME, el

cual carece hasta la fecha de una adaptación parecida. El objetivo es trasladar este enfoque de interpretabilidad semántica al funcionamiento de LIME, creando variantes que sustituyan los métodos clásicos por otros que los combinen con la lógica difusa y el *Soft Computing*.

3.2. Lógica Difusa y Soft Computing

La Lógica Difusa o Borrosa (*Fuzzy Logic*) es una extensión de la lógica clásica que proporciona un marco matemático riguroso para modelar la incertidumbre, la imprecisión y el razonamiento aproximado. Esta teoría surge como respuesta a las limitaciones de la lógica clásica o booleana, la cual enuncia que cualquier proposición debe ser estrictamente verdadera o falsa (valores de verdad restringidos al conjunto discreto $\{0, 1\}$).

En la realidad, y especialmente en la comprensión para las personas, las fronteras conceptuales rara vez son nítidas. La Lógica Difusa flexibiliza esta rigidez permitiendo que los valores de verdad se representen mediante grados de pertenencia continuos en el intervalo cerrado $[0, 1]$, donde 0 representa la falsedad absoluta, 1 la verdad absoluta, y cualquier valor intermedio hace referencia a una verdad parcial. Esto se consigue mediante el uso de reglas **SI-ENTONCES** (*IF-THEN*) difusas.

Este mecanismo de reglas **SI-ENTONCES** es fundamental, ya que permite que un modelo de explicabilidad no solo asigne un peso a una variable, sino que construya un relato lógico y comprensible de cómo la interacción de múltiples características desemboca en una decisión concreta. Esta arquitectura de inferencia será la base de los Sistemas de Inferencia Difusos detallados en el siguiente apartado.

3.2.1. Conjuntos Difusos y Funciones de Pertenencia

El pilar fundamental de este tipo de lógica es el conjunto difuso. Sea X un espacio o universo de discurso y x un elemento genérico de X. Un conjunto difuso A en X se define matemáticamente mediante una función de pertenencia (*Membership Function*), denotada como $\mu_A(x)$, que asocia a cada elemento x un número real:

$$\mu_A : X \to [0, 1]$$

El conjunto difuso A se representa como un conjunto de pares ordenados que relacionan cada elemento con su grado de pertenencia:

$$A = \{(x, \mu_A(x)) \mid x \in X\}$$

La forma de la función $\mu_A(x)$ determina el comportamiento del conjunto. En implementaciones de ingeniería, las funciones de pertenencia más utilizadas por su eficiencia computacional son las triangulares, las trapezoidales y las gaussianas. Un ejemplo de esta última se puede ver en la siguiente figura 3.1.

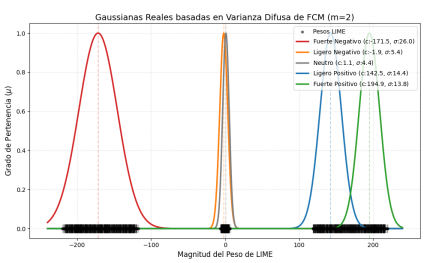


Figura 3.1: Conjuntos difusos con función de pertenencia gaussiana

3.2.2. Variables Lingüísticas

Para que un sistema difuso sea interpretable, se introdujo el concepto de variable lingüística. A diferencia de las variables numéricas tradicionales, una variable lingüística toma como valores palabras o sentencias en lenguaje natural (denominados términos lingüísticos). Por ejemplo, si $x =$ "Temperatura", su conjunto de términos podría ser $T(x) = \{\text{BAJA}, \text{MEDIA}, \text{ALTA}\}$. Cada uno de estos términos se modela internamente como un conjunto difuso independiente sobre el dominio numérico (por ejemplo, grados Celsius). Esta abstracción es el puente semántico que permite traducir magnitudes indescifrables a diagnósticos comprensibles.

3.2.3. Sistemas de Inferencia Difusos

Un Sistema de Inferencia Difuso (FIS, por sus siglas en inglés *Fuzzy Inference System*, también conocido como Sistema Basado en Reglas Borrosas) es un proceso computacional que emula el mecanismo de toma de decisiones humano mediante la aplicación de la lógica difusa. Su función es establecer un mapeo no lineal desde un espacio de variables de entrada hacia un espacio de variables de salida, operando sobre conocimiento modelado en lenguaje natural.

Arquitectura de un FIS

Estructuralmente, un Sistema de Inferencia Difuso se compone de cuatro bloques:

1. **Base de Conocimiento:** Es el repositorio de la información del sistema. Se subdivide en dos componentes principales:
   - *Diccionario lingüístico:* Contiene las definiciones matemáticas de las funciones de pertenencia (triangulares, trapezoidales, etc.) asociadas a las etiquetas lingüísticas de cada variable.
   - *Base de Reglas:* Contiene el conjunto de proposiciones lógicas del tipo SI-ENTONCES (*IF-THEN*) que modelan el comportamiento del sistema.
2. **Fuzzificador:** Es la interfaz de entrada. Se encarga de transformar los valores numéricos exactos que son rígidos (*crisp*) en grados de pertenencia a los conjuntos difusos definidos en el diccionario lingüístico.
3. **Mecanismo de Inferencia:** Es el núcleo de procesamiento. Evalúa qué reglas de la base de conocimiento se activan dados los valores fuzzificados de entrada y computa la inferencia matemática para deducir un conjunto difuso de salida.
4. **Defuzzificador:** Es la interfaz de salida. Puesto que las aplicaciones del mundo real (o la evaluación de un modelo subrogado) normalmente requieren un valor numérico escalar y no un conjunto difuso, este bloque se encarga de extraer un valor rígido (*crisp*) representativo del conjunto difuso resultante de la inferencia.

Propiedades deseables de los modelos difusos

Uno de los objetivos del modelado difuso, al igual que en cualquier otra técnica de modelado, es conseguir capacidad aproximativa. Sin embargo, su característica principal es la capacidad de incorporar conocimiento experto y explicar el funcionamiento de un sistema mediante reglas de tipo SI-ENTONCES. Este tipo de modelos nos permite comprender la influencia de cada variable de entrada en la salida, proporcionando la transparencia e interpretabilidad [MER03].

Cuando construimos estos modelos desde un experto humano a través de reglas lingüísticas, la transparencia es una propiedad inherente. Sin embargo, cuando este conocimiento es incompleto

o inexistente y nos vemos obligados a extraer las reglas a partir de datos de entrada y salida, la interpretabilidad deja de ser automática y suele perderse, es decir, los modelos mejoran su capacidad aproximativa a costa de reducir su capacidad descriptiva. Por este motivo podríamos preguntarnos sobre qué aspectos son necesarios para la interpretabilidad de un modelo difuso.

Para ello existe el mecanismo FC^3, el cual establece que un sistema de reglas difusas debe cumplir con cuatro propiedades esenciales: flexibilidad, completitud, consistencia y compactitud [MER03].

- **Flexibilidad:** Hace referencia a la instanciación de los operadores difusos que aparecen en las reglas, lo que influye de manera directa en la capacidad de aproximación del sistema.
- **Completitud:** Garantiza que el sistema sea capaz de derivar una salida adecuada para cualquier entrada posible. Esta propiedad la dividimos, a su vez, en la completitud de la partición difusa (asegurando que se cubra todo el rango de cada variable) y la completitud de la estructura de las reglas (comprobando que todos los conjuntos difusos definidos intervengan en, al menos, una regla del sistema) [MER03].
- **Consistencia:** Evita la aparición de contradicciones en la base de conocimientos. Un sistema pierde consistencia si detectamos dos o más reglas con antecedentes muy similares pero con consecuentes bastante diferentes, o si surgen conflictos directos con el conocimiento experto preexistente.
- **Compactitud:** Mide la simplicidad del modelo. Un sistema resulta mucho más transparente y asimilable para el usuario cuanto menor sea su número de reglas, el número de variables en sus antecedentes y la cantidad de funciones de pertenencia empleadas.

Por otro lado, en el trabajo [OLI99], Oliveira propone algunas restricciones para asegurar que las funciones de pertenencia individuales sean interpretables por un operador humano [MER03]:

- **Distinguibilidad:** Cada función de pertenencia debe representar un término lingüístico con un significado claro y diferenciado de los demás conjuntos vecinos.
- **Normalidad:** Para que una etiqueta semántica tenga un significado claro, debe existir al menos un elemento en el universo de discurso cuyo grado de pertenencia sea igual a 1.
- **Número moderado de funciones:** Oliveira limita la cantidad de etiquetas por variable a un rango de entre 7 y 9 dimensiones, respetando los límites de la memoria activa definidos por la Psicología Cognitiva.
- **Cubrimiento:** Es el principio de completitud de las funciones de pertenencia tratado anteriormente.

El Proceso de Inferencia (Modelo de Mamdani)

El proceso de evaluación de un FIS, asumiendo la arquitectura clásica de Mamdani, sigue una secuencia matemática estricta para evaluar una entrada en forma de vector $\mathbf{x} = (x_1, x_2, \ldots, x_n)$:

1. Cálculo del grado de disparo (Fuerza del antecedente) Para cada regla i de la base de reglas, se evalúan las condiciones de su antecedente. Si el antecedente está compuesto por múltiples variables unidas por la conjunción Y (AND), se aplica una T-norma (habitualmente el operador mínimo) para obtener el grado de disparo w_i:

$$w_i = \text{mín}(\mu_{A_{i1}}(x_1), \mu_{A_{i2}}(x_2), \ldots, \mu_{A_{in}}(x_n)) \tag{3.4}$$

2. Implicación difusa El grado de disparo w_i determina el peso que tendrá el consecuente de la regla i en la decisión final. Mediante el método de implicación del mínimo (truncamiento), la función de pertenencia del conjunto difuso de salida original de la regla (C_i) se recorta a la altura de w_i, generando un nuevo conjunto de salida modificado C_i':

$$\mu_{C_i'}(y) = \text{mín}(w_i, \mu_{C_i}(y)) \tag{3.5}$$

3. Agregación Dado que el sistema evalúa todas las reglas en paralelo, se obtienen múltiples conjuntos de salida truncados $C_1', C_2', \ldots, C_k'$. La agregación combina todos estos conjuntos parciales en un único conjunto difuso global de salida C_{final} utilizando una T-conorma (habitualmente la del máximo):

$$\mu_{C_{final}}(y) = \text{máx}(\mu_{C_1'}(y), \mu_{C_2'}(y), \ldots, \mu_{C_k'}(y)) \tag{3.6}$$

4. Métodos de Defuzzificación El último paso es condensar el conjunto C_{final} en un único número real y^*. La literatura [MER25c] define varios métodos analíticos para este cálculo, siendo los más destacados:

- **Centro de Gravedad o Centroide (COG - *Center of Gravity*):** Es el método más utilizado y robusto. Calcula el centro de masas del área bajo la curva del conjunto agregado. Para un universo de discurso continuo, se define como:

$$y^*_{COG} = \frac{\int y \cdot \mu_{C_{final}}(y)\, dy}{\int \mu_{C_{final}}(y)\, dy} \tag{3.7}$$

- **Media Ponderada de los Centros (*Weighted Average*):** Este método supone una alternativa computacionalmente más eficiente que el cálculo del Centroide clásico. En lugar de resolver la integral del área global agregada, determina el valor escalar de salida ponderando el centro geométrico de cada conjunto difuso del consecuente por su respectivo grado de activación. Matemáticamente se define mediante la siguiente expresión:

$$y^*_{WA} = \frac{\sum_{i=1}^{m} \overline{y}^i \cdot \mu_i(\overline{y}^i)}{\sum_{i=1}^{m} \mu_i(\overline{y}^i)} \tag{3.8}$$

  donde m es el número total de reglas activadas, $\overline{y}^i$ representa el valor central de la función de pertenencia i, y $\mu_i(\overline{y}^i)$ es su grado de activación correspondiente.

Modelos FIS Alternativos: Takagi-Sugeno (TSK)

Mientras que el modelo de Mamdani utiliza conjuntos difusos tanto en el antecedente como en el consecuente (priorizando la semántica y la interpretabilidad), una de las principales alternativas es el modelo de **Sugeno**. [SUG85]

En un FIS de tipo Sugeno, los antecedentes siguen siendo difusos, pero el consecuente de la regla es una función matemática determinista de las variables de entrada (comúnmente un polinomio lineal constante o de primer orden, por ejemplo: $y = a \cdot x_1 + b \cdot x_2 + c$). La defuzzificación se reduce a una media ponderada simple, lo que lo hace computacionalmente muy eficiente y apto para problemas de optimización o control adaptativo.

Sin embargo, para los propósitos de este TFM, el modelo de Sugeno en principio no nos resultará de utilidad, ya que sacrifica la interpretabilidad lingüística del consecuente. Sin embargo, nos podría servir como concepto para combinar la salida proporcionada por Mamdani

(de manera lingüística) con el valor real asociado a dicha etiqueta, de modo que obtengamos un valor rígido (*crisp*) y su etiqueta lingüística en la salida de las explicaciones de las distintas instancias.

3.2.4. Soft Computing

Dentro del paradigma del *Soft Computing*, las técnicas de agrupamiento (*clustering*) juegan un papel fundamental para el descubrimiento de estructuras ocultas en los datos. En esta sección se verán distintas técnicas de *clustering* a nivel teórico, además de un algoritmo clásico de aprendizaje como lo es un árbol de decisión.

Método de Clustering Sustractivo (SCM)

Una de las técnicas más destacadas en este ámbito es el Método de Clustering Sustractivo propuesto originalmente por Chiu (1994) [CHI94]. Tal y como se detalla en [MER03], este algoritmo nace como una evolución del Método de Clustering de Montaña (MCM).

Esta estrategia considera a los propios puntos del conjunto de datos como los únicos candidatos a ser centros de clúster. De esta forma, el coste computacional pasa a depender únicamente del número total de instancias[MER03].

El algoritmo comienza evaluando el potencial de cada punto como posible centro del clúster. Para ello, calcula una "función montaña" para cada dato z_i, definida según la siguiente fórmula:

$$M(z_i) = \sum_{k=1}^{n} e^{-\frac{||z_k - z_i||^2}{(\frac{r_a}{2})^2}}$$

donde r_a es una constante positiva que define el radio de vecindad. Como se explica en [MER03], la influencia de los puntos z_k sobre el candidato z_i que están fuera de ese radio r_a tienen una influencia mínima.

Una vez seleccionado un clúster, se debe revisar el potencial del resto de puntos x_i para eliminar su efecto. Esto se realiza mediante la siguiente ecuación:

$$M_j(z_i) = M_{j-1}(z_i) - M_{j-1}^{*} e^{-\frac{||z_{j-1}^{*} - z_i||^2}{(r_b/2)^2}}$$

Donde r_b es otro radio de vecindad que determina que los puntos más cercanos al clúster seleccionado (z_{j-1}^{*}) tendrán una mayor reducción en su potencial [MER03].

Por último, también se incluye un criterio de terminación más sofisticado. En lugar de utilizar un umbral fijo, el SCM emplea un intervalo $[\epsilon_{min}, \epsilon_{max}]$. Si el potencial máximo es superior a ϵ_{max}, el punto se selecciona como clúster. Si está por debajo de ϵ_{min}, se rechaza y el algoritmo termina. Para valores intermedios, se evalúa un compromiso entre el potencial del candidato y su distancia a los clústeres ya existentes.

Sin embargo, como se ve en [MER03], el SCM es más sensible al ruido y su precisión depende de que los centros "reales" de los clústeres coincidan con algún punto del conjunto de datos original.

Clustering Fuzzy C-Means

El algoritmo Fuzzy C-Means (FCM) es un método de clustering difuso que permite que cada dato pertenezca simultáneamente a varios clústers con distintos grados de pertenencia. El objetivo del algoritmo es minimizar una función de coste basada en la distancia entre los datos y los centroides de los clústers [MER03].

La formulación general del algoritmo introduce tres elementos principales:

- Una matriz de pertenencia difusa U, donde cada valor indica el grado de pertenencia de un dato a un clúster.
- Un parámetro de difusividad m, que controla el nivel de borrosidad de la partición. Cuando m aumenta, los clústers son más difusos.
- Una medida de distancia determinada por una matriz A, que permite utilizar distintas métricas para calcular la similitud entre datos.

El algoritmo FCM funciona de manera iterativa. En cada iteración se realizan tres pasos principales:

1. Cálculo de los centroides de los clústers.
2. Cálculo de las distancias entre cada dato y los centroides.
3. Actualización de la matriz de pertenencia difusa.

El proceso se repite hasta que la diferencia entre dos iteraciones consecutivas sea menor que un umbral de terminación [MER03].

Este algoritmo cuenta con varios parámetros importantes:

- **Parámetro de terminación** (ϵ): determina el criterio de parada del algoritmo.
- **Parámetro de fuzziness** (m): controla el grado de solapamiento entre clústers. Valores cercanos a 1 producen agrupaciones más definidas, mientras que valores mayores generan clústers más difusos.
- **Matriz inicial** (U^0): la elección de la matriz de pertenencia inicial influye en el resultado final, ya que el algoritmo puede converger a óptimos locales.
- **Número de clústers** (c): debe fijarse previamente y condiciona la estructura final de la partición.
- **Matriz de distancia** (A): determina el tipo de distancia utilizada para medir similitudes entre datos. Las tres medidas de distancia más utilizadas son:
  - **Distancia euclídea**: genera clústers con forma hiperesférica.
  - **Distancia diagonal**: produce clústers hiperelipsoidales alineados con los ejes de coordenadas.
  - **Distancia de Mahalanobis**: permite obtener clústers hiperelipsoidales con orientaciones arbitrarias.

Sin embargo, la elección de la distancia utilizada puede imponer estructuras concretas sobre los datos aunque estas no representen adecuadamente su distribución real. Además, la utilización de ciertas métricas, como la distancia euclídea, requiere normalizar previamente los datos para evitar efectos negativos sobre el resultado del algoritmo [MER03].

Árboles de decisión

Un árbol de decisión es un algoritmo de aprendizaje supervisado utilizado tanto para tareas de clasificación como de regresión. Su estructura tiene forma de árbol jerárquico compuesto por un nodo raíz, ramas, nodos internos y nodos hoja. Cada nodo representa una decisión basada en una característica de los datos, y las hojas representan el resultado final obtenido [EDA25].

El funcionamiento de un árbol de decisión consiste en dividir los datos en subconjuntos cada vez más homogéneos según determinadas condiciones. Este proceso se realiza de manera recursiva hasta alcanzar un criterio de parada. Debido a su estructura visual, los árboles de decisión suelen ser fáciles de interpretar y permiten entender de forma clara cómo se ha llegado a una determinada predicción [EDA25].

Tipos de árboles de decisión

Existen varios algoritmos distintos de árboles de decisión, en concreto veremos los siguientes:

- **ID3**: utiliza la entropía y la ganancia de información para seleccionar las mejores divisiones de los datos.
- **C4.5**: es una evolución del algoritmo ID3 y puede emplear tanto la ganancia de información como la razón de ganancia para construir el árbol.
- **CART (*Classification and Regression Trees*)**: se utiliza para problemas de clasificación y regresión y emplea normalmente la impureza de Gini para determinar las divisiones más adecuadas.

Ventajas e inconvenientes

Entre las principales ventajas de los árboles de decisión destacan las siguientes:

- Son fáciles de interpretar gracias a su representación visual y lógica.
- Requieren poca preparación de datos y pueden trabajar con valores continuos, categóricos e incluso datos incompletos.
- Son flexibles, ya que pueden utilizarse tanto para clasificación como para regresión.

Sin embargo, también presentan algunos inconvenientes:

- Son propensos al sobreajuste, especialmente cuando el árbol crece demasiado y se adapta en exceso a los datos de entrenamiento.
- Tienen alta varianza, ya que pequeñas variaciones en los datos pueden generar árboles muy diferentes.
- El proceso de entrenamiento puede resultar costoso debido a la búsqueda de las mejores divisiones en cada nodo [EDA25].

3.2.5. Estado actual de la Lógica Difusa y el Soft Computing

En esta sección, veremos la situación actual en la que se encuentra el *Soft Computing*, explorando ciertas investigaciones que se han llevado a cabo en los últimos años y exponiendo el problema que presenta este paradigma en la actualidad (el invierno del *Soft Computing*).

Investigaciones actuales

A pesar de las idas y venidas en la popularidad de los algoritmos de Inteligencia Artificial, la concepción de que el *Soft Computing* o la Lógica Difusa son tecnologías obsoletas es errónea. En la actualidad, este paradigma sigue siendo muy relevante además de ser un área de investigación bastante activa en la literatura científica.

Una revisión de las publicaciones recientes en revistas como IEEE demuestra que estas técnicas se siguen usando en dominios bastante complejos. Por ejemplo, la lógica difusa se sigue aplicando en el desarrollo de controladores adaptativos para robótica móvil y evasión de obstáculos [IEEE1], en el control de sistemas no lineales con restricciones asimétricas [IEEE2], en la predicción dinámica de trayectorias físicas [IEEE3], e incluso en el diseño de algoritmos avanzados de agrupamiento (*clustering*) basados en particiones granulares [IEEE4].

Sin embargo, el uso más destacable de la lógica difusa en la actualidad para este trabajo está en el ámbito de la Inteligencia Artificial Explicable (XAI). Ante el problema de la "caja negra" asociado a los modelos de Aprendizaje Profundo (*Deep Learning*), los investigadores están recurriendo de nuevo a la lógica difusa para hacer a estos sistemas más interpretables.

Un ejemplo es el trabajo de Hou et al. [IEEE5], quienes proponen el uso de Mapas Cognitivos Difusos mejorados (CAE-FCM) para la predicción de series temporales multivariantes. Su arquitectura logra mantener una alta precisión predictiva mientras ofrece explicabilidad gracias a las relaciones causales que el modelo ha aprendido.

También encontramos un estudio desarrollado para el diagnóstico de fallos industriales [IEEE6]. En este trabajo, los autores combinan Redes Neuronales Profundas (DNN) con Redes Neuronales Difusas (FNN) mediante técnicas de destilación de conocimiento. Los investigadores extraen el conocimiento complejo de un modelo opaco de alto rendimiento (el *teacher*) y lo transfieren a un sistema basado en la arquitectura de Takagi-Sugeno-Kang que utiliza reglas lingüísticamente interpretables. Esto demuestra que para que los modelos modernos puedan ser explicados por expertos humanos de una manera más sencilla, la semántica de la lógica difusa facilita mucho esta tarea.

El invierno del Soft Computing

Durante las últimas décadas, la Inteligencia Artificial ha experimentado un cambio de paradigma marcado por la supremacía del Aprendizaje Profundo (*Deep Learning*). Este dominio no ha casualidad, sino que es el resultado de un gran desarrollo tecnológico: la disponibilidad de volúmenes masivos de datos (*Big Data*) y el enorme desarrollo de las capacidades de hardware, especialmente mediante el uso de Unidades de Procesamiento Gráfico (GPUs) y clústeres de supercomputación.

Gracias a esto, arquitecturas complejas como las redes neuronales profundas han logrado mejorar su precisión en tareas complejas para las máquinas, tales como la visión por computador, el procesamiento del lenguaje natural o el reconocimiento de voz. Sin embargo, este auge ha provocado un efecto en la comunidad investigadora: el desplazamiento de otras ramas de la inteligencia artificial, dando lugar a lo que metafóricamente llamamos "el invierno del *Soft Computing*".

El *Soft Computing* vivió su época dorada a finales del siglo XX. Su principal virtud está

en su capacidad para modelar la incertidumbre, tolerar la imprecisión y, sobre todo, emular el razonamiento humano de forma transparente. Sin embargo, ante a la fuerza bruta computacional del *Deep Learning* y su gran precisión sobre conjuntos de datos enormes, las técnicas puras de *Soft Computing* fueron desplazadas a un segundo plano.

El coste de la precisión: La crisis de la opacidad

A pesar de su éxito predictivo, el *Deep Learning* y los modelos de *ensembles* (como *Random Forest* o *Gradient Boosting*) han demostrado una gran desventaja: **la no explicabilidad**.

Estos modelos operan como verdaderas "cajas negras". Sus decisiones se basan en millones, o incluso miles de millones de parámetros matemáticos no lineales. La estructura de estas redes es tan compleja que es prácticamente imposible de entender matemáticamente para una persona.

En entornos académicos, esta opacidad puede ser aceptable a cambio de una alta precisión. Sin embargo, en la aplicación real de la Inteligencia Artificial en dominios como el diagnóstico médico, la concesión de créditos financieros o la conducción autónoma, confiar ciegamente en las predicciones de una caja negra resulta ética y legalmente inviable.

Es precisamente esta crisis de opacidad la que ha demostrado que la precisión por sí sola ya no es suficiente. El sistema necesita transparencia y justificabilidad, unos requisitos que han servido como idea principal para un nuevo campo de investigación: el uso del *Soft Computing* como herramienta para ayudar a explicar estos modelos complejos.

3.3. Propuestas de combinación entre XAI y Soft Computing

Tras la revisión teórica y del estado del arte de los paradigmas de XAI y *Soft Computing*, en esta sección se presentan distintas combinaciones entre ambos enfoques con el objetivo de conseguir la posible sinergia existente entre ellos.

Por parte del paradigma XAI, se ha decidido utilizar la herramienta LIME debido a que, como se ha comentado anteriormente, no se han encontrado implementaciones que integren técnicas propias del *Soft Computing*. En cuanto a las técnicas de *Soft Computing*, se han seleccionado tres métodos diferentes: el uso de árboles de decisión para obtener un FIS, el algoritmo de clustering *Fuzzy C-Means* y el método de clustering *Subtractive Clustering*.

A partir de estas combinaciones se plantean tres herramientas distintas:

- Una primera herramienta en la que el modelo lineal interpretable utilizado por LIME se sustituye por un sistema de reglas difusas obtenido a partir de árboles de decisión.
- Una segunda herramienta orientada a generar rankings similares a los producidos por LIME, pero utilizando conjuntos difusos en lugar de valores numéricos reales.
- Una tercera herramienta basada en *Subtractive Clustering*, cuyo objetivo es seleccionar un conjunto reducido de instancias representativas de forma autónoma que permita explicar el comportamiento global del modelo de caja negra.

Estas propuestas buscan explorar nuevas formas de generar explicaciones interpretables mediante la combinación de técnicas clásicas de XAI con métodos propios del *Soft Computing*.

Capítulo 4

Implementación

4.1. Contextualización

4.1.1. Enfoques de Explicabilidad Difusa Propuestos

En este trabajo se proponen tres enfoques distintos que integran la lógica difusa en el entorno de LIME, cada una diseñada para resolver problemas específicos de la Inteligencia Artificial Explicable (XAI), como la rigidez de los modelos lineales o la inestabilidad de las explicaciones locales.

Fuzzy-LIME

Fuzzy-LIME es una herramienta innovadora en el ámbito de la Inteligencia Artificial Explicable (XAI) que fusiona las perturbaciones de LIME (*Local Interpretable Model-agnostic Explanations*) con la interpretabilidad de la Lógica Difusa (*Fuzzy Logic*).

Mientras que el algoritmo LIME original busca aproximar el comportamiento de un modelo de "caja negra" (como una red neuronal o un *Random Forest*) generando un modelo sustituto interpretable (típicamente una regresión lineal) en un enfoque local, **Fuzzy-LIME sustituye este modelo lineal por un sistema de reglas difusas**. En lugar de devolver un conjunto de coeficientes numéricos asociados a cada variable, Fuzzy-LIME genera explicaciones basadas en variables lingüísticas y grados de pertenencia, siguiendo la estructura de los sistemas de inferencia de Mamdani. (por ejemplo: *"SI la variable X es ALTA Y la variable Y es MEDIA, ENTONCES la predicción es BAJA"*).

FuzzyRank-LIME

El enfoque **FuzzyRank-LIME** surge para solucionar la inestabilidad de LIME. Debido al muestreo aleatorio que realiza el algoritmo, una única ejecución puede ofrecer resultados ligeramente distintos. Para solucionarlo, FuzzyRank-LIME propone la ejecución iterativa (N veces) de LIME sobre una misma instancia o conjunto de ellas para construir un *universo de discurso* de explicaciones.

A partir de este histórico de pesos, se aplica *Fuzzy C-Means* para fuzzificar la importancia de las variables. Este proceso genera funciones de pertenencia Gaussianas que permiten etiquetar lingüísticamente la importancia de cada variable. Este método puede aplicarse tanto a explicaciones locales como a la selección global de *Submodular Pick* (SP-LIME), dando a las importancias un significado semántico estable dentro del modelo.

SC-LIME

Finalmente, **SC-LIME** propone una modificación del método de selección de instancias representativas para la comprensión global del modelo. Tradicionalmente, SP-LIME utiliza un algoritmo codicioso de optimización submodular para elegir K instancias que maximicen la "cobertura" de variables, obligando al usuario a predefinir el número de ejemplos deseados.

En su lugar, SC-LIME introduce el uso de **Subtractive Clustering (SC)** como método de selección basado en densidad. Al analizar el espacio de las explicaciones generadas, el algoritmo identifica de forma autónoma los centroides de la lógica del modelo. Esto permite obtener un conjunto de instancias representativas sin necesidad de forzar un número fijo de ejemplos (K), evitando incluir instancias redundantes u omitir explicaciones relevantes del modelo, logrando así una visión global mucho más fiel al modelo.

4.1.2. Justificación del uso de Reglas Difusas frente a un Explicador Lineal

La sustitución del clásico modelo de regresión lineal de LIME por un sistema de reglas difusas puede aportar una serie de ventajas tanto a nivel computacional como cognitivo:

- **Captura de no-linealidades locales:** Un modelo lineal asume que, en una vecindad pequeña, la relación entre las entradas y la salida describe un hiperplano lineal. Sin embargo, frente a modelos subyacentes complejos, la aproximación lineal puede fracasar o perder fidelidad. Las reglas difusas pueden capturar estas no-linealidades de forma mucho más precisa.

- **Fronteras de decisión suaves (*Soft Boundaries*):** A diferencia de las reglas rígidas (*crisp*) extraídas por un árbol de decisión tradicional (por ejemplo, $X > 2{,}5$), la lógica difusa permite transiciones graduales (por ejemplo, X *es ALTO*). Esto refleja de una manera más cómoda y comprensible cómo los modelos de caja negra, que basan sus salidas en probabilidades o funciones de activación continuas, toman decisiones en la realidad, evitando así saltos en las explicaciones ante alteraciones mínimas en los datos de entrada.

- **Alta interpretabilidad cognitiva:** Los coeficientes de una regresión lineal (por ejemplo, $\beta_1 = 152{,}41$) pueden resultar abstractos y difíciles de contextualizar, especialmente si las variables tienen escalas distintas. Las reglas difusas abstraen la complejidad numérica y explican el comportamiento utilizando la intuición humana mediante etiquetas semánticas (BAJO, MEDIO, ALTO)

- **Robustez ante el ruido y la incertidumbre:** La lógica difusa está diseñada para manejar la imprecisión. En el contexto de vecindarios generados sintéticamente (como realiza LIME mediante la perturbación de instancias), un modelo difuso gestiona el ruido ligado a este proceso de muestreo de manera más eficiente que un modelo lineal rígido.

4.1.3. Entrenamiento del modelo de caja negra

Para este trabajo, se ha seleccionado como modelo de caja negra el modelo ***Random Forest***. La elección de este algoritmo es debido a su propia naturaleza: al tratarse de un método de ensamblado (*ensemble*) que promedia las predicciones de varios de árboles de decisión independientes, el modelo es capaz de capturar relaciones no lineales y ofrecer un alto rendimiento predictivo. Sin embargo, esta misma agregación evita la interpretabilidad. Esta pérdida de transparencia convierte al *Random Forest* en un ejemplo clásico de modelo opaco.

4.2. Diseño e implementación del dataset sintético

Para evaluar de manera rigurosa y objetiva cada uno de nuestros enfoques, hemos decidido aislar los experimentos iniciales de las impurezas, valores atípicos y ruidos ligados a los conjuntos de datos del mundo real. Por este motivo, el primer paso en nuestra implementación consiste en crear un entorno de pruebas controlado mediante la generación de datos sintéticos.

El uso de bases de datos generadas de forma artificial nos permite ajustar la complejidad del espacio de características y establecer una línea base de comparación. Para analizar el comportamiento de los distintos enfoques frente a diferentes datos de entrada, la implementación de este dataset sintético se ha estructurado en dos escenarios de evaluación complementarios:

- **Polinomio conocido:** Un primer escenario basado en una función matemática fija y predefinida. Este enfoque permite analizar, trazar y depurar manualmente cómo nuestros enfoques capturan curvas e interacciones concretas entre variables que han sido introducidas a propósito. La variable objetivo *target* para cada instancia se calcula mediante la siguiente función polinómica $f(X)$:

$$target = f(x_1, x_2, \ldots, x_6) = -1 * x_1^2 + 2 * x_2^1 - 3 * x_3^5 + 3 * x_4^5 + 1 * x_5^2 - 2 * x_6^1 \quad (4.1)$$

  donde $X = (x_1, x_2, \ldots, x_6)$.

- **Polinomio aleatorio:** Un segundo escenario orientado a medir la capacidad de generalización y la robustez del método. Al generar funciones polinómicas con coeficientes e interacciones aleatorias, se evalúa el rendimiento del explicador frente a cualquier distribución de los datos. La variable objetivo *target* queda definida por el siguiente sumatorio:

$$target = f_{rand}(X) = \sum_{i=1}^{d} c_i \cdot x_i^{p_i} \quad (4.2)$$

  Donde $c_i \in \mathbb{Z} \cap [-10, 10]$ y $p_i \in \mathbb{Z} \cap [1, 4]$.

Además, para conseguir una validación real de las herramientas, se ha hecho uso del dataset *California Housing Dataset* que busca predecir el valor de una casa en California.

4.3. Fuzzy-LIME

Tras haber contextualizado las limitaciones algorítmicas de los explicadores locales tradicionales (como la fragmentación del conocimiento en SP-LIME y la rigidez de los modelos lineales), y habiendo definido el entorno experimental, esta sección aborda el diseño, desarrollo e implementación de **Fuzzy-LIME**.

Si bien en secciones anteriores se justificó a nivel teórico la conveniencia de combinar la lógica difusa con la Inteligencia Artificial Explicable (XAI), este apartado propone una posible solución. Fuzzy-LIME no se concibe únicamente como una modificación aislada de LIME, sino como un *pipeline* o flujo de trabajo modular, lo cual se aprecia en la figura 4.1:

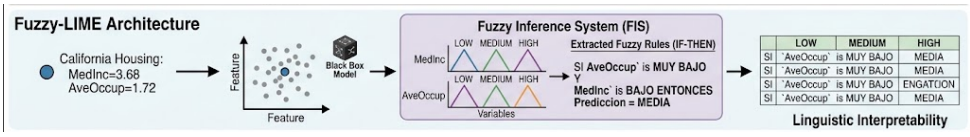


Figura 4.1: Esquema de Fuzzy-LIME

Su objetivo principal es actuar como un puente semántico: inferir la complejidad de un modelo predictor opaco y devolver un sistema de razonamiento global, continuo y fácil de interpretar para los usuarios.

Para lograr esta transición de lo numérico a lo lingüístico, la arquitectura de Fuzzy-LIME ha sido diseñada como un proceso secuencial compuesto por varias etapas, las cuales se detallarán a lo largo de esta sección:

- **Extracción local:** Captura de las fronteras de decisión en vecindarios específicos utilizando aproximadores interpretables.
- **Fuzzificación del espacio:** Mapeo de los valores numéricos y umbrales rígidos hacia variables lingüísticas continuas mediante funciones de pertenencia y diccionarios de contexto.
- **Traducción y agregación:** Resolución de conflictos entre reglas contradictorias y síntesis del conocimiento mediante técnicas de reducción de reglas, consolidando las explicaciones locales en un único modelo global.
- **Motor de Inferencia:** Proceso donde las reglas extraídas operan bajo la arquitectura de un motor tipo Mamdani para generar nuevas explicaciones.

A continuación, se explican cada una de estas fases, sentando las bases de este enfoque que posteriormente será encapsulado en la librería de uso público detallada en la sección 4.4.

4.3.1. Extracción de reglas

El primer paso en esta propuesta consiste en capturar la lógica de decisión del modelo de "caja negra" en un entorno local y traducirla a un formato simbólico inicial.

Para ello, la primera modificación consiste en sustituir el regresor lineal de LIME por un modelo subrogado que sea interpretable y basado en reglas, capaz de realizar particiones del espacio de características en lugar de trazar un único hiperplano.

Árbol de decisión

Para lograr la extracción de estas reglas lógicas de forma automatizada, se utiliza un Árbol de Decisión (*Decision Tree Regressor*) como modelo subrogado local. El proceso de integración de este modelo se desarrolla en las siguientes etapas:

1. Intercepción del vecindario sintético (Función "Espía")
Se utiliza una función *wrapper* o "función espía" para interceptar las llamadas de LIME al modelo. Esta técnica permite capturar y almacenar las instancias perturbadas (Z) y las predicciones de la caja negra ($f(Z)$) sin modificar la librería original, obteniendo así el vecindario local exacto sobre el que se genera la explicación.

2. Entrenamiento del subrogado basado en reglas
Una vez capturado el vecindario local, se procede a entrenar el Árbol de Decisión sobre los pares de datos $(Z, f(Z))$. En comparación con LIME, la estructura del árbol modela mejor relaciones locales complejas mediante particiones sucesivas del espacio de características. La profundidad de este árbol se controla mediante hiperparámetros para evitar el *overfitting* y mantener la interpretabilidad.

3. Extracción de rutas (Reglas *Crisp*)
Las rutas del árbol desde la raíz hasta las hojas se transforman en reglas condicionales del tipo *SI-ENTONCES*. Estas reglas *crisp* representan el comportamiento local del modelo de caja negra, aunque todavía presentan rigidez debido a sus umbrales estrictos, lo que nos lleva al siguiente paso, la fuzzificación.

4.3.2. Fuzzificación del vocabulario

Las reglas *crisp* extraídas en la fase anterior tienen una alta fidelidad local, pero son poco intuitivas debido a sus umbrales numéricos estrictos. Para mejorar su interpretabilidad, Fuzzy-LIME aplica un proceso de **fuzzificación**, que transforma variables numéricas continuas en conceptos lingüísticos mediante la Teoría de Conjuntos Difusos. Cada variable se representa usando funciones de pertenencia (μ) definidas sobre su universo de discurso.

Vocabulario de entrada

El vocabulario de entrada modela las variables independientes del conjunto de datos mediante conjuntos difusos. Para cada variable, se define un universo de discurso a partir de sus valores mínimos y máximos, sobre el que se establecen cinco etiquetas lingüísticas: *MUY BAJO, BAJO, MEDIO, ALTO* y *MUY ALTO*.

Estas etiquetas se representan mediante funciones de pertenencia triangulares y trapezoidales solapadas, permitiendo transiciones suaves entre conceptos. La estructura resultante permite traducir umbrales numéricos del árbol de decisión a grados de pertenencia difusos $\mu \in [0, 1]$.

Vocabulario de salida

El vocabulario de salida fuzzifica la predicción del modelo de caja negra para dar significado lingüístico al consecuente de las reglas difusas. Se construye de forma análoga al vocabulario de entrada, definiendo un universo de discurso sobre las predicciones y cinco etiquetas lingüísticas: *MUY BAJA, BAJA, MEDIA, ALTA* y *MUY ALTA*.

Mientras el vocabulario de entrada se utiliza para interpretar condiciones, el de salida participa activamente en el proceso de defuzzificación, permitiendo convertir las reglas activadas de nuevo a una predicción numérica continua.

4.3.3. Traductor Lingüístico

El Traductor Lingüístico transforma las reglas *crisp* del árbol de decisión en reglas semánticas basadas en lógica difusa. Para ello, combina los umbrales numéricos de las reglas con los vocabularios difusos definidos previamente.

El proceso comienza consolidando todas las restricciones asociadas a una misma variable en un único intervalo numérico. Posteriormente, se calcula un valor representativo de dicho intervalo y se evalúa su grado de pertenencia respecto a las etiquetas lingüísticas disponibles. La etiqueta con mayor activación se selecciona como representación semántica de la condición.

El mismo procedimiento se aplica a la predicción del árbol utilizando el vocabulario de salida. Finalmente, las condiciones se ordenan y concatenan mediante el operador lógico *AND*, obteniendo reglas interpretables en lenguaje natural del tipo:

$$x_1 \textit{ es BAJO Y } x_2 \textit{ es MEDIO} \rightarrow \textit{Predicción MEDIA}$$

4.3.4. Agregación global de reglas difusas

Una vez finalizado el proceso de traducción lingüística, Fuzzy-LIME dispone de múltiples reglas difusas locales que deben consolidarse para construir un explicador global coherente. La fuzzificación favorece esta agregación, ya que distintas reglas numéricas similares convergen en las mismas etiquetas semánticas.

El proceso comienza agrupando las reglas según su antecedente y consecuente, calculando la frecuencia con la que aparece cada una. Esta frecuencia actúa como medida de relevancia global, indicando qué patrones son más representativos del comportamiento del modelo.

Posteriormente, se resuelven posibles conflictos entre reglas con la misma condición pero diferentes predicciones. Para ello, se aplica un criterio de mayoría representativa: se conserva únicamente la regla con mayor frecuencia y se descartan las variantes minoritarias.

El resultado es un conjunto único de reglas difusas consistente que constituye la base del explicador global final.

No obstante, en conjuntos de datos de grandes dimensiones, el número de reglas resultantes aún puede ser inmanejable, requiriendo una última fase de compresión mediante técnicas de reducción de reglas.

4.3.5. Reducción de la Base de Reglas

Para comprimir este conocimiento global en un número reducido y manejable de "reglas maestras", Fuzzy-LIME aplica una fase de reducción basada en la frecuencia de aparición de las reglas, denominada **Poda por Conjunto Soporte**. Su objetivo es conservar únicamente las reglas más representativas del modelo.

Para cada regla se calcula un soporte estadístico, definido como la proporción de instancias locales explicadas por dicha regla respecto al total. Formalmente:

$$S(r_i) = \frac{f(r_i)}{N_{total}} \tag{4.3}$$

donde $f(r_i)$ es la frecuencia absoluta de aparición de la regla r_i.

Las reglas cuyo soporte es inferior a un umbral mínimo α (por ejemplo, $\alpha = 0{,}02$, equivalente al 2 %) se consideran poco relevantes y se eliminan de la base de conocimiento. Finalmente, las reglas restantes se ordenan según su soporte, obteniendo un conjunto compacto, robusto y fácilmente interpretable.

Validación de las Reglas Maestras (El impacto de la no-linealidad)

Para comprobar la validez de la fase de entrenamiento, vamos a analizar el conjunto final de Reglas Maestras devuelto por el sistema y contrastarlo con la función polinómica que generó los datos. En este experimento, el conjunto de datos sintético fue diseñado introduciendo términos de alta no-linealidad, elevando específicamente las variables x_3 y x_4 a la quinta potencia (x^5).

Al imprimir por pantalla el modelo global difuso resultante (`df_modelo_final`) obtenemos la siguiente salida:

Condición	Prediccion_Linguistica
x3 es MUY BAJO Y x4 es MUY BAJO	Prediccion_Linguistica = BAJA
x3 es MEDIO Y x4 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x3 es MUY BAJO Y x4 es MEDIO	Prediccion_Linguistica = ALTA
x3 es ALTO Y x4 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x3 es MUY BAJO Y x4 es ALTO	Prediccion_Linguistica = MUY ALTA
x3 es MUY ALTO Y x4 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x3 es ALTO Y x4 es MEDIO	Prediccion_Linguistica = BAJA
x3 es MUY BAJO Y x4 es MUY ALTO	Prediccion_Linguistica = MUY ALTA
x3 es ALTO Y x4 es ALTO	Prediccion_Linguistica = MEDIA
x3 es ALTO Y x4 es BAJO	Prediccion_Linguistica = BAJA
x3 es MUY BAJO Y x4 es BAJO	Prediccion_Linguistica = ALTA
x3 es ALTO Y x4 es MUY ALTO	Prediccion_Linguistica = MEDIA
x3 es MEDIO Y x4 es MEDIO	Prediccion_Linguistica = MEDIA

Figura 4.2: Reglas Difusas generadas tras el entrenamiento

Al analizar semánticamente estas reglas, se comprueba cómo la metodología de Fuzzy-LIME ha logrado capturar con éxito el funcionamiento del modelo de "caja negra". Una variable de grado 5 (x_i^5) influye bastante el valor de salida frente a variables lineales (x_j^1) a medida que nos alejamos del origen.

Este comportamiento ha sido extraído por el algoritmo y plasmado en las reglas de forma intuitiva ya que dichas variables de grado 5 son las dos únicas variables que aparecen en las reglas.

4.3.6. Inferencia Difusa

Una vez consolidada la base de conocimiento global, Fuzzy-LIME utiliza las reglas como un modelo predictivo mediante un Motor de Inferencia Difuso de Mamdani. Este sistema permite procesar nuevas instancias de datos y generar tanto una explicación lingüística como una predicción numérica.

El proceso comienza fuzzificando la entrada, calculando el grado de pertenencia de cada variable a sus etiquetas lingüísticas. Después, las reglas difusas se evalúan aplicando la T-norma del mínimo sobre las condiciones conectadas por *AND*s, obteniendo así la fuerza de activación de cada regla.

Si varias reglas activan la misma salida lingüística, sus resultados se agregan utilizando la T-conorma del máximo. La etiqueta con mayor activación se toma como predicción lingüística final.

Por último, el sistema realiza la defuzzificación mediante el cálculo del centroide de las funciones de pertenencia activadas, obteniendo una predicción numérica continua comparable con la del modelo original.

La ejecución de este predictor difuso no solo devuelve un valor predictivo, sino que proporciona una lista con las reglas específicas que se activaron y su peso porcentual en la decisión final.

4.3.7. Ejemplo de uso

Para validar el correcto funcionamiento de todas las fases descritas anteriormente, realizaremos una prueba práctica. En esta prueba, se selecciona una instancia aislada perteneciente al conjunto de datos de validación `X_test` (la cual nunca ha sido vista por el modelo durante su entrenamiento) y se evalúa tanto por la caja negra original (nuestro modelo `RandomForest`) como por el explicador difuso de Fuzzy-LIME.

Puesto que vamos a realizar una simple comprobación del funcionamiento de nuestra herramienta, la salida de la inferencia se extrae imprimiendo por pantalla el estado de las variables y los resultados de la defuzzificación.

Para ello extraemos y formateamos en texto plano los tres elementos clave de la Inteligencia Artificial Explicable: el valor numérico predicho, la justificación semántica y el desglose de las reglas que motivaron la decisión. El resultado obtenido es el siguiente:

```
================================================
 INSTANCIA DE TEST #0
================================================
Predicción original Caja Negra (RF): -49.14
Predicción oficial Fuzzy-LIME:       >>> BAJA <<<
   *(Valor numérico inferido: -96.40)*

 ¿POR QUÉ? (Explicación del Modelo):
  [Fuerza: 100.00%] -> SI x3 es MUY BAJO Y x4 es MUY BAJO ENTONCES Prediccion = BAJA
```

Análisis del resultado

Este ejemplo muestra el funcionamiento real de esta propuesta y expone el compromiso (*trade-off*) ligado a la explicabilidad global. En términos puramente numéricos, se observa una diferencia evidente entre la predicción exacta de la caja negra (−49,14) y el valor *crisp* inferido por Fuzzy-LIME (−96,40). Esta desviación es una consecuencia esperable: al comprimir un modelo subyacente altamente complejo en etiquetas lingüísticas discretas, se sacrifica precisión matemática local en favor de la generalización. El valor inferido de −96,40 corresponde al centro de masas de la etiqueta "BAJA", el cual arrastra la predicción numérica hacia ese punto del universo de salida.

Sin embargo, el verdadero valor añadido de este enfoque es la transparencia del proceso de decisión. Mientras que el modelo original (*Random Forest*) predice el valor −49,14 de forma completamente opaca, Fuzzy-LIME traduce la decisión a una explicación semánticamente coherente ("BAJA"). Además, el explicador expone el porqué de esta decisión, demostrando con un 100 % de fuerza de activación que la predicción está determinada por la regla "SI x3 es MUY BAJO Y x4 es MUY BAJO ENTONCES Prediccion = BAJA".

4.4. API (Librería) de Fuzzy-LIME

A pesar de que la herramienta funciona correctamente, la ejecución mediante celdas independientes de un notebook (`.ipynb`) y la lectura en consola mediante texto plano carece de la profesionalidad necesaria para su uso por parte de usuarios finales. Esta limitación es la que motiva el desarrollo de esta sección, donde toda la herramienta se refactoriza y encapsula en una librería de `Python`.

4.4.1. Creación de la librería

La herramienta que hemos diseñado se ha integrado en una librería orientada a objetos con el objetivo de facilitar su uso en entornos de *Machine Learning*. La arquitectura sigue la estructura habitual de librerías como `scikit-learn` o el propio `lime`, incluyendo métodos familiares como `fit()` y `explain_instance()`.

La librería se organiza en dos clases principales: `FuzzyLimeExplainer`, que implementa toda la lógica del modelo, y `ExplicacionDifusa`, encargada de la representación visual de los resultados.

Clase ExplicacionDifusa

La clase `ExplicacionDifusa` almacena la información generada por el explicador, incluyendo la predicción lingüística, el valor numérico y las reglas activadas. Además, incorpora el método `show_in_notebook()`, que genera una interfaz HTML/CSS para visualizar las explicaciones de forma intuitiva mediante barras de progreso (coloreadas en verde, amarillo o rojo) que representan visualmente la fuerza de activación de cada regla disparada.

Clase FuzzyLimeExplainer

La clase `FuzzyLimeExplainer` es el núcleo de la librería e integra todas las etapas del sistema: extracción de reglas, fuzzificación, agregación, poda e inferencia de Mamdani.

Su método `fit()` entrena el modelo global y construye la base de reglas difusas, mientras que `explain_instance()` procesa nuevas instancias aplicando el motor de inferencia difuso y devuelve una explicación interpretable mediante un objeto `ExplicacionDifusa`.

4.4.2. Uso de la librería

El resultado final de esta refactorización es un flujo de trabajo mucho más limpio. Lo que en etapas anteriores requería ejecutar y gestionar múltiples celdas de código y variables globales, ahora se consolida en tres sencillos pasos, ofreciendo una experiencia de uso similar a la de LIME.

Esta simplicidad en el uso, combinada con la interfaz gráfica de los resultados, consolida a Fuzzy-LIME no solo como un experimento válido, sino como una herramienta de *software* con potencial de ser usada para explicar modelos de aprendizaje automático en situaciones reales.

Validación y Visualización de Resultados

Para finalizar, vamos a realizar una prueba de validación, comprobando que la refactorización del código no ha alterado la fidelidad del modelo subyacente y demostrando una posible mejora en la experiencia de usuario.

Al instanciar la clase `FuzzyLimeExplainer` y ejecutar el método `explain_instance()` sobre una nueva instancia del conjunto de prueba, se aprecia una mejora en la calidad gracias a la salida proporcionada por la clase `ExplicacionDifusa`. La llamada al método `show_in_notebook()` renderiza una interfaz gráfica directamente en el entorno de trabajo, tal y como se muestra en la Figura 4.3, sustituyendo la necesidad de inspeccionar reglas en texto plano.

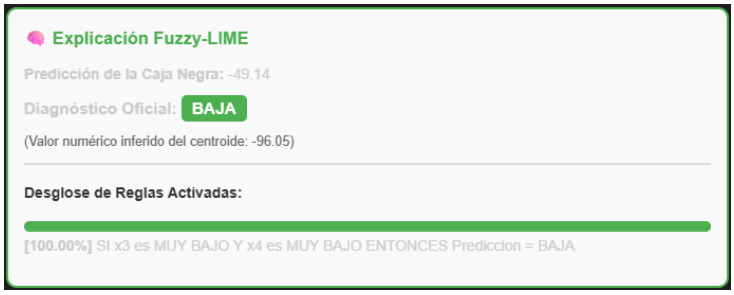


Figura 4.3: Interfaz visual generada por la API de Fuzzy-LIME en un entorno Jupyter.

Análisis de la Interfaz Visual

Como se puede observar en la captura de la ejecución (Figura 4.3), la salida gráfica está diseñada para mejorar la transparencia y minimizar la carga cognitiva del usuario:

- **Contexto predictivo:** Se muestra en la cabecera tanto la predicción numérica cruda de la caja negra ($-49{,}14$) como el valor numérico inferido por el modelo difuso ($-96{,}05$).
- **Diagnóstico Oficial:** La etiqueta lingüística ganadora (*BAJA*) se resalta como la decisión principal del sistema, informando al usuario del comportamiento del modelo en lenguaje natural de un simple vistazo.
- **Trazabilidad Visual (Desglose de Reglas):** Las proposiciones lógicas que motivaron la decisión se muestran en forma de lista. Para facilitar su interpretación, el *software* implementa unas barras de progreso horizontales ordenadas por color según su relevancia. En este ejemplo, el usuario identifica visualmente de forma inmediata que la decisión está definida por una regla principal (con una fuerza de activación del 100 %, mostrada en color verde).

Análisis Comparativo del Agrupamiento

A modo de comparativa, vamos a ver la explicación de una instancia modificando el parámetro `min_soporte` donde este toma los valores `0` y `0.02` respectivamente. Esta comparativa la realizamos sobre la instancia 9 del conjunto `X_test` para poder apreciar con más detalle como es la visualización con más de una regla activa y cómo cambian las activaciones de dichas reglas cuando no se ha aplicado el mecanismo de poda.

La figura 4.4 muestra una comparativa entre la salida con las reglas originales (Figura 4.4a) y la salida habiendo aplicado la reducción mediante Poda por Soporte (Figura 4.4b).



(a) Sin Poda por Soporte



(b) Con Poda por Soporte

Figura 4.4: Explicación visual de la instancia 9

Gracias a esta comparativa podemos apreciar lo siguiente:

1. **Preservación de la fidelidad predictiva:** El dato más destacable de la comparativa es que el valor numérico inferido por el centroide de Mamdani permanece igual (68,62) en ambos escenarios, coincidiendo en la predicción lingüística (*MEDIA*). Esto demuestra que el proceso de poda es capaz de comprimir la base de conocimiento global sin sacrificar la precisión local durante la inferencia.

2. **Reducción de la carga cognitiva:** En el modelo sin compresión (Figura 4.4a), la interfaz presenta cuatro proposiciones lógicas. Sin embargo al aplicar la compresión (Figura 4.4b), una de estas reglas es eliminada, presentando al usuario una salida más limpia y asimilable: una regla dominante verde (87,41 %) que compite con dos reglas secundarias (12,59 % y 9,07 %).

En resumen, la creación de esta API no solo consolida la integración entre el algoritmo LIME y la Lógica Difusa, sino que proporciona a este Trabajo de Fin de Máster un *software* funcional en la práctica para proporcionar explicaciones de modelos de caja negra.

4.5. FuzzyRank-LIME

El enfoque **FuzzyRank-LIME** nace como respuesta a uno de los problemas de la Inteligencia Artificial Explicable: la **inestabilidad de las explicaciones locales**. Puesto que LIME se basa en un muestreo aleatorio alrededor de la instancia a explicar, pequeñas variaciones en la semilla de generación pueden derivar en cambios relativamente significativos en los coeficientes numéricos de la explicación. Esta falta de robustez puede dificultar la confianza del usuario final en el sistema.

4.5.1. Fundamentación y Metodología de Implementación

FuzzyRank-LIME propone una arquitectura basada en la agregación de conocimiento y la representación difusa del impacto de las variables. Esta arquitectura se divide en tres fases:

1. **Generación de un Universo de Discurso Iterativo:** En lugar de confiar en una única ejecución del `explainer`, esta metodología realiza la **ejecución iterativa de LIME (**N **veces)** sobre una misma instancia. Este proceso genera una *pool* o base de datos de pesos de importancia (en nuestro caso $N = 300$). De esta manera, la importancia de una variable deja de ser un valor puntual para convertirse en una *distribución de probabilidad* del impacto real.

2. **Calibración del Diccionario Lingüístico Global:** Una vez obtenido el histórico de pesos, se utiliza el algoritmo *Fuzzy C-Means* (FCM) para identificar los centros de masa de las importancias. A diferencia de un uso tradicional de clustering, aquí FCM se emplea para descubrir de forma autónoma los niveles de importancia naturales del modelo (p. ej., dónde empieza a considerarse un impacto como "Alto" o "Neutro").

   A partir de estos centroides, se construyen **Funciones de Pertenencia Gaussianas** cuya anchura (σ) se parametriza mediante la varianza de los datos y el coeficiente de *fuzzyness* (m). El resultado es un "Diccionario Lingüístico" estable que particiona el eje de importancia en etiquetas semánticas.

3. **Traducción y Mapeo en Tiempo de Inferencia:** Con el diccionario calibrado, cualquier nueva explicación (una ejecución $N + 1$) puede ser traducida instantáneamente. Los pesos numéricos realizan un proceso de *fuzzificación*, donde se calcula su grado de pertenencia (μ) a cada etiqueta del diccionario.

Esta arquitectura descrita se resume en la imagen 4.5:

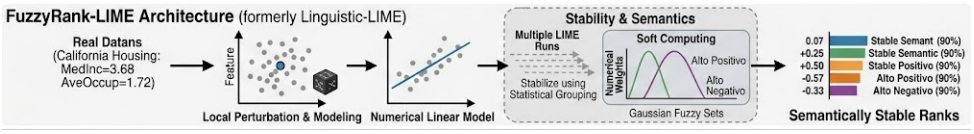


Figura 4.5: Esquema de FuzzyRank-LIME

4.5.2. Justificación del uso del Enfoque Lingüístico

Desde un punto de vista conceptual, FuzzyRank-LIME ofrece dos ventajas principales frente al uso de LIME estándar:

- **Estabilidad Semántica:** Al promediar el conocimiento de múltiples ejecuciones, el sistema es capaz de filtrar el "ruido" del muestreo de LIME, garantizando que la etiqueta asignada (p. ej., "Fuerte Positivo") sea coherente y robusta frente a cambios mínimos en los datos.
- **Reducción de la Carga Cognitiva:** Se elimina la necesidad de que el analista humano interprete coeficientes con múltiples decimales (como 0,3421 vs 0,3458). En su lugar, el sistema ofrece una interpretación basada en el lenguaje natural, permitiendo una comunicación más fluida entre la lógica del modelo y el usuario final.

Esta metodología es extensible a la interpretación global de modelos mediante su integración con *Submodular Pick* (SP-LIME). En este escenario, FuzzyRank-LIME permite que las instancias más representativas del modelo no se presenten como gráficos de barras numéricos, sino como etiquetas lingüísticas que facilitan la comparación entre diferentes zonas del espacio de decisión.

4.6. SC-LIME

Mientras que las variantes anteriores se centran más en la interpretación de predicciones individuales, **SC-LIME** (*Subtractive Clustering LIME*) aborda el desafío de la explicabilidad global. El objetivo es proporcionar al usuario una visión global del funcionamiento del modelo de caja negra mediante la selección de un conjunto de instancias representativas que resuman su comportamiento general.

A modo conceptual, este enfoque se resume con la figura 4.6:

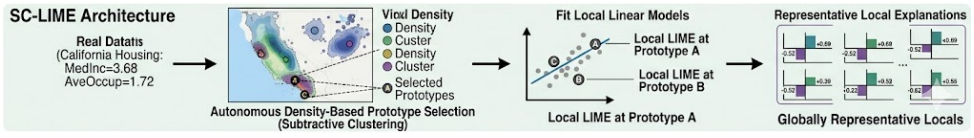


Figura 4.6: Esquema de SC-LIME

4.6.1. Limitaciones de la Selección Submodular Convencional

Tradicionalmente, el estándar para la explicabilidad global es el algoritmo *Submodular Pick* (SP-LIME). Este método emplea una optimización codiciosa (*greedy*) para seleccionar K instancias que maximicen la "cobertura" de las variables, asegurando que se muestren ejemplos que utilicen el mayor número posible de características distintas.

Sin embargo, este enfoque presenta dos inconvenientes principales:

- **Rigidez del parámetro K:** Obliga al usuario a predefinir el número de explicaciones. Si K es demasiado bajo, se pierde información; si es demasiado alto, el algoritmo se ve forzado a incluir instancias redundantes o poco informativas.
- **Sensibilidad a valores atípicos:** Al buscar "cobertura" a toda costa, SP-LIME puede seleccionar instancias extremas (*outliers*) simplemente porque utilizan variables poco comunes, ignorando así las tendencias principales del modelo.

4.6.2. Fundamentación y Metodología de SC-LIME

SC-LIME sustituye la métrica de cobertura submodular por un análisis de densidad en el espacio de las explicaciones utilizando el algoritmo de **Clustering Sustractivo** (Chiu, 1994) [CHI94]. La implementación se divide en los siguientes pasos:

1. **Construcción de la Matriz de Explicaciones:** Se genera una muestra representativa de explicaciones locales de LIME, extrayendo sus vectores de pesos numéricos. Estos vectores se normalizan para asegurar que la métrica de distancia sea consistente a través de todas las dimensiones del espacio de características. Pese a que en nuestro caso es conveniente normalizar esta matriz, no siempre es así, ya que como indica Robert Babuška [BAB96], hay situaciones donde normalizar supondría separar demasiado los clústers y por lo tanto obtendríamos valores que no pertenecen a ningún conjunto difuso, generando de esta manera peores resultados.
2. **Cálculo del Potencial de Densidad:** Para cada explicación en el espacio, se calcula un potencial de densidad exponencial. Este valor depende del número de explicaciones vecinas dentro de un radio de influencia (r_a). Las zonas con mayor acumulación de explicaciones similares generan los picos de densidad más altos, identificando los comportamientos principales del modelo.
3. **Selección Iterativa y Sustracción de Potencial:** El algoritmo selecciona la instancia con el mayor potencial actual como la primera explicación global. Acto seguido, se resta una fracción del potencial de todas las instancias en su vecindad (utilizando un radio de penalización r_b). Este proceso se repite de forma iterativa, limpiando las zonas ya explicadas y buscando el siguiente centro de masa natural.
4. **Determinación Autónoma del Número de Prototipos:** El proceso finaliza cuando el potencial del mejor candidato restante cae por debajo de un umbral de aceptación relativo. De este modo, el sistema decide de forma autónoma cuántas explicaciones son necesarias para representar fielmente el modelo.

4.6.3. Justificación del uso del Análisis Basado en Prototipos

La principal aportación de SC-LIME es su capacidad para identificar la estructura real de la lógica del modelo. En lugar de buscar una diversidad artificial basada en variables, SC-LIME busca la representatividad basada en la frecuencia de comportamiento. Si el modelo se comporta de la misma manera para el 90 % de los datos, SC-LIME devolverá un único prototipo dominante, mientras que en modelos con fronteras de decisión complejas y heterogéneas, el sistema escalará automáticamente el número de explicaciones para capturar cada bloque de decisión detectado.

Capítulo 5

Experimentación

5.1. Evaluación de Fuzzy-LIME

Para validar las ventajas teóricas planteadas en los capítulos anteriores, realizaremos una comparativa entre las explicaciones generadas por la implementación base de LIME y las obtenidas mediante la arquitectura propuesta en este trabajo, **Fuzzy-LIME**.

5.1.1. Análisis Local: Explicación de una instancia individual

En esta fase, el objetivo es evaluar la interpretabilidad y la capacidad de síntesis de ambas herramientas sobre una predicción concreta. Para este experimento, hemos seleccionado la Instancia 0, donde el modelo de caja negra (*Random Forest*) ha obtenido una predicción de $-49,14$.

Explicación mediante LIME Clásico

Al aplicar LIME estándar (véase la Figura 5.1), la herramienta intenta aproximar la predicción del modelo de caja negra mediante un modelo lineal local. La explicación se muestra como un desglose de importancias positivas y negativas para cada una de las seis variables del sistema:

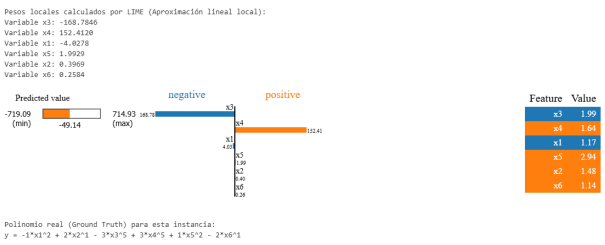


Figura 5.1: Explicación de LIME sobre la instancia 0

Como se observa, LIME presenta la información en un formato puramente cuantitativo. Aunque permite identificar que x_3 y x_4 son las variables con mayor peso, obliga al analista a interpretar una lista completa de seis coeficientes decimales de magnitudes muy distintas. El usuario debe procesar mentalmente esta suma de pesos para comprender la lógica del modelo, lo que supone una carga cognitiva elevada, especialmente cuando se busca una validación rápida en entornos con muchas dimensiones.

Explicación mediante Fuzzy-LIME

En contraste, nuestra arquitectura **Fuzzy-LIME** simplifica esta interpretación al centrar la

atención en las variables importantes y utilizar semántica lingüística (véase la Figura 5.2). Para esta misma instancia, el sistema genera la siguiente explicación difusa:

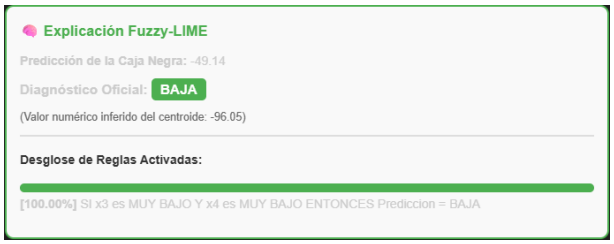


Figura 5.2: Explicación de Fuzzy-LIME sobre la instancia 0

Los resultados de la ejecución confirman la potencia de este enfoque: la regla se dispara con un **grado de activación del** 100 % y obtiene un valor numérico defuzzificado de $-96,05$. Es importante destacar que Fuzzy-LIME ha identificado correctamente que el comportamiento del modelo para esta instancia depende principalmente de la interacción entre x_3 y x_4. Al filtrar las otras cuatro variables que aportaban pesos de menor importancia en LIME (x_1, x_5, x_2 y x_6), logramos eliminar el ruido y ofrecer una explicación mucho más clara.

Conclusión de la comparativa local

Esta comparativa muestra la superioridad cognitiva de Fuzzy-LIME frente al LIME original. Mientras que LIME clásico devuelve una tabla de seis pesos numéricos que deben ser analizados uno a uno, nuestra propuesta condensa la lógica del modelo en una sola sentencia lógica comprensible. Al transformar los valores crudos en conceptos semánticos (como "MUY BAJO"), Fuzzy-LIME permite que un experto comprenda la decisión del modelo de caja negra de forma inmediata, sin la necesidad de evaluar coeficientes matemáticos complejos.

5.1.2. Análisis Global: Comprensión general del modelo

Más allá del análisis de predicciones aisladas, vamos a intentar conseguir la "confianza en el modelo" (*trusting a model*). Para ello, es necesario comprender el comportamiento de la caja negra en su totalidad. En esta fase, comparamos la solución global propuesta por LIME clásico (mediante el uso de la técnica *Submodular Pick LIME* o SP-LIME) frente al diccionario de reglas globales extraído por nuestra arquitectura **Fuzzy-LIME**.

Explicación Global mediante SP-LIME

Por su parte, el algoritmo SP-LIME selecciona un subconjunto de instancias representativas (en nuestro caso, cinco explicaciones globales) que, en teoría, resumen el comportamiento del modelo de regresión, como se puede apreciar en la Figura 5.3.

Al analizar los resultados, observamos que cada una de las cinco explicaciones devuelve un análisis completo de las seis variables numéricas. Por ejemplo, la **Explicación 1** (con una predicción de $-187,06$) indica que x_3 supone un impacto negativo muy fuerte (con un peso de $185,34$) y x_4 un impacto positivo fuerte (con un peso de $129,78$), mientras que x_6, x_2, x_5 y x_1 muestran pesos marginales (inferiores a $2,00$). Este patrón se repite en las otras cuatro explicaciones representativas: en la **Explicación 2** (con una predicción de $490,12$), x_4 empuja la predicción positivamente con un peso de $203,23$ y x_3 negativamente con un peso de $121,31$.

Aunque es preciso, este enfoque somete a la persona encargada de analizar los datos a una enorme carga cognitiva. Para extraer el conocimiento global, el usuario debe leer cinco tablas distintas, comparar mentalmente treinta coeficientes numéricos de distintas magnitudes, ignorar

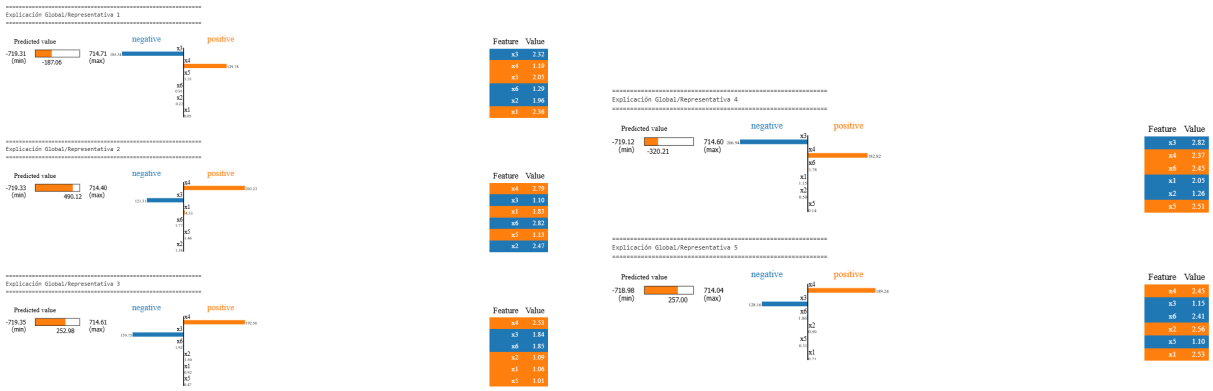


Figura 5.3: Explicación global generada por SP-LIME

el ruido de las variables irrelevantes (x_1, x_2, x_5, x_6) y deducir por sí mismo la relación inversa entre x_3 y x_4.

Explicación Global mediante Fuzzy-LIME

Por el contrario, la aplicación de **Fuzzy-LIME** a nivel global automatiza este proceso de deducción y abstracción. El sistema filtra las variables que solo aportan ruido a la información y condensa toda la lógica del *Random Forest* en un diccionario limpio de tan solo trece reglas lingüísticas, como se aprecia en la Figura 5.4.

Condición	Prediccion_Linguistica
x3 es MUY BAJO Y x4 es MUY BAJO	Prediccion_Linguistica = BAJA
x3 es MEDIO Y x4 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x3 es MUY BAJO Y x4 es MEDIO	Prediccion_Linguistica = ALTA
x3 es ALTO Y x4 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x3 es MUY BAJO Y x4 es ALTO	Prediccion_Linguistica = MUY ALTA
x3 es MUY ALTO Y x4 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x3 es ALTO Y x4 es MEDIO	Prediccion_Linguistica = BAJA
x3 es MUY BAJO Y x4 es MUY ALTO	Prediccion_Linguistica = MUY ALTA
x3 es ALTO Y x4 es ALTO	Prediccion_Linguistica = MEDIA
x3 es ALTO Y x4 es BAJO	Prediccion_Linguistica = BAJA
x3 es MUY BAJO Y x4 es BAJO	Prediccion_Linguistica = ALTA
x3 es ALTO Y x4 es MUY ALTO	Prediccion_Linguistica = MEDIA
x3 es MEDIO Y x4 es MEDIO	Prediccion_Linguistica = MEDIA

Figura 5.4: Reglas generadas por Fuzzy-LIME

Como podemos observar, el conjunto de reglas obtenido cumple todas las propiedades que debe de tener un modelo de inferencia difusa, a excepción de la completitud de la estructura de las reglas, probablemente debido a la poda que se ha realizado y a que hay variables que no son importantes en nuestros datos.

Además, este conjunto de reglas demuestra coherencia con los números de SP-LIME, pero presentados en un formato directamente asimilable por el cerebro humano. Al observar la segunda regla, el sistema confirma explícitamente el patrón deducido anteriormente: cuando la variable de impacto negativo (x_3) es muy alta y la de impacto positivo (x_4) es muy baja, el resultado global baja (predicción **MUY BAJA**). Esto también sucede a la inversa, como se observa en la cuarta regla, si x_3 es muy bajo y x_4 es alto, la predicción es **MUY ALTA**.

Conclusión de la comparativa global

Los resultados obtenidos en este análisis global reafirman la premisa central del trabajo. SP-LIME es una herramienta potente para seleccionar qué instancias explicar, pero falla en la forma de presentar esa información conjunta, delegando en el usuario la tarea de buscar patrones entre decenas de valores decimales.

La hibridación lograda en **Fuzzy-LIME** resuelve este inconveniente. Al abstraer el problema, pasamos de un análisis tedioso a un manual de instrucciones semántico de trece líneas. De esta manera, el experto obtiene al momento una visión panorámica, precisa y humanizada de cómo podría razonar el modelo de caja negra en todo el espacio de características.

5.1.3. Evaluación con un dataset distinto

Para validar la robustez y la capacidad de generalización de la arquitectura de Fuzzy-LIME, a continuación realizamos una evaluación utilizando un nuevo conjunto de datos sintético. En los siguientes apartados se detallan los resultados obtenidos tanto a nivel de interpretabilidad local como global, contrastando el desempeño de Fuzzy-LIME frente a la implementación original de LIME.

Evaluación a Nivel Local

Empezaremos con la evaluación a nivel local utilizando un conjunto de datos sintético no lineal. El modelo de caja negra (*Random Forest*) fue entrenado para aproximar la siguiente función polinómica:

$$y = 2x_1^3 - 9x_2^2 + 1x_3^1 + 7x_4^1 + 1x_5^3 - 7x_6^4 \tag{5.1}$$

El análisis se ha centrado en la **instancia 6** del conjunto de test. Para esta instancia, la predicción real del modelo de caja negra es de $-93{,}69$. Si analizamos la Ecuación 5.1, es evidente que el término $-7x_6^4$ es el causante de este valor fuertemente negativo, ya que el valor elevado de x_6 (1,90) elevado a la cuarta potencia domina por completo sobre el resto de variables.

Análisis de la Explicación con LIME Clásico

Al aplicar el algoritmo LIME estándar sobre esta predicción (véase la Figura 5.5), el método genera un modelo lineal subrogado local. LIME identifica correctamente que la variable x_6 es la que empuja la predicción hacia valores negativos, asignándole el mayor peso absoluto ($-18{,}18$). Además, reparte pesos positivos de menor magnitud entre el resto de variables, destacando x_4 (+6,68) y x_1 (+4,93).

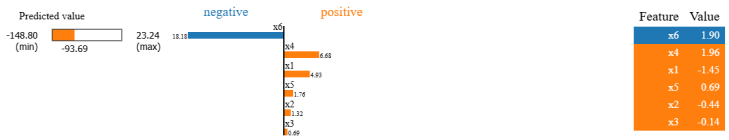


Figura 5.5: Explicación de la instancia 6 del conjunto de test hecha por LIME

Sin embargo, podemos ver como esta explicación sigue teniendo una **alta carga cognitiva** para el usuario. Se le devuelve un gráfico de seis coeficientes numéricos que intentan aproximar una curva de grado 4 mediante una recta. El usuario debe realizar un análisis para deducir qué variables son verdaderamente determinantes y cuáles son simplemente ruido residual en esa región de los datos.

Análisis de la Explicación con Fuzzy-LIME

Por el contrario, la aplicación de la herramienta Fuzzy-LIME sobre la misma instancia (Figura 5.6) ofrece un cambio notable. El motor de inferencia difuso clasifica el diagnóstico general como **"MUY BAJA"**, prediciendo un valor numérico de $-103{,}71$, una aproximación muy cercana a la predicción original de la caja negra ($-93{,}69$), demostrando una alta fidelidad local.

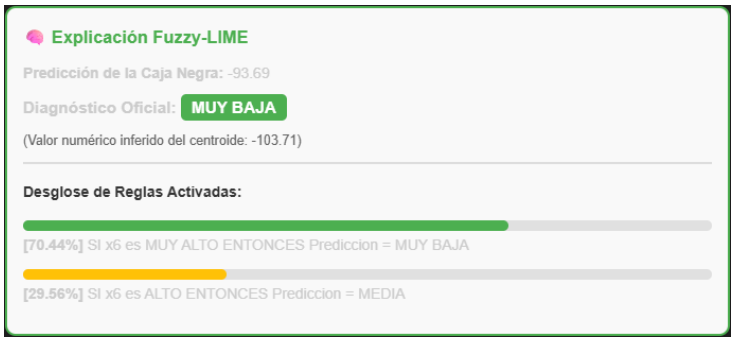


Figura 5.6: Explicación de la instancia 6 del conjunto de test

El aspecto más destacable está en el desglose de reglas activadas. El sistema de poda y evaluación lingüística muestra que la explicación se basa en una única variable:

- **[70.44 %]** SI x_6 es MUY ALTO ENTONCES Predicción = MUY BAJA.
- **[29.56 %]** SI x_6 es ALTO ENTONCES Predicción = MEDIA.

Fuzzy-LIME logra abstraer la presencia de las variables x_1, x_2, x_3, x_4 y x_5. Al evitar la rigidez de los coeficientes lineales, el sistema comprende que, en este caso, la decisión está definida únicamente por x_6. La explicación lingüística obtenida capta la no linealidad del dataset: el valor $x_6 = 1{,}90$ activa fuertemente el conjunto difuso "MUY ALTO", lo cual obtiene como consecuente "MUY BAJA", ajustándose al comportamiento del término $-7x_6^4$ de la ecuación original.

Este resultado demuestra que Fuzzy-LIME no solo reduce es gran medida la carga cognitiva al traducir números crudos a lenguaje natural, sino que actúa como un filtro de variables secundarias, aislando verdadero núcleo lógico de la decisión del modelo.

Evaluación a Nivel Global

Una vez demostrada la eficacia de la herramienta a nivel local, es necesario evaluar su capacidad para abstraer el comportamiento global del modelo.

Análisis de la Explicación Global con SP-LIME De nuevo, haremos uso del algoritmo *Submodular Pick* (SP-LIME). Al forzar al algoritmo a seleccionar $K = 5$ instancias representativas (véase la Figura 5.7), el sistema devuelve cinco explicaciones locales distintas.

Si bien SP-LIME cumple su trabajo de maximizar la cobertura de variables, el resultado necesita de nuevo un análisis profundo por parte del usuario. Para comprender el modelo subyacente, el analista debe examinar treinta coeficientes numéricos distribuidos en cinco escenarios distintos. Aunque un análisis exhaustivo revela que x_6 y x_4 suelen presentar los pesos absolutos más altos en la mayoría de los casos, el modelo subrogado también asigna pesos considerables a variables que en la realidad tienen un impacto menor, como x_1 o x_2.

Además, al ser aproximaciones lineales locales, SP-LIME es incapaz de mostrar explícitamente el comportamiento no lineal de la función. El usuario no tiene forma de saber, a simple vista, que los extremos de x_6 reducen la predicción, ya que solo ve pendientes (pesos) positivas o negativas aisladas según el punto de muestreo.

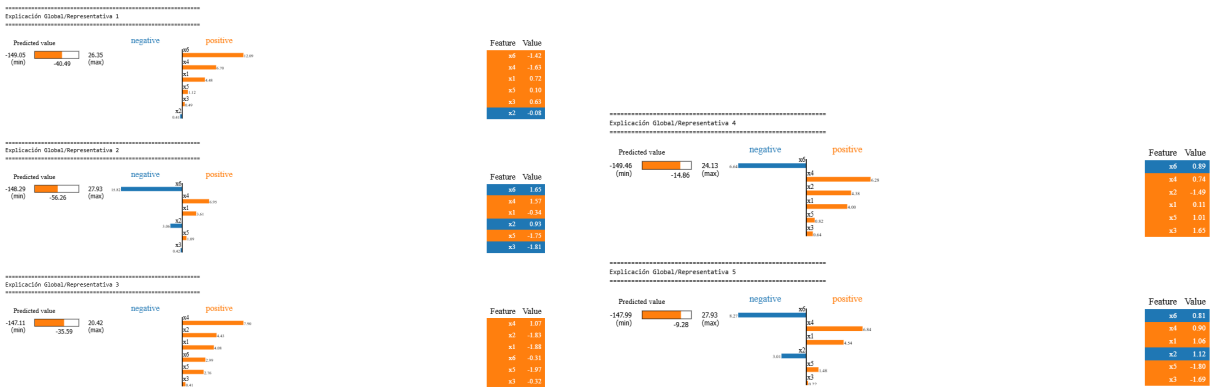


Figura 5.7: Explicación global generada por SP-LIME con otro polinomio

Análisis de la Explicación Global con Fuzzy-LIME Por otro lado, la arquitectura Fuzzy-LIME procesa todas las explicaciones generadas y, mediante la técnica de Poda por Conjunto Soporte, resume el comportamiento global del modelo en un conjunto reducido de reglas lingüísticas (véase la Figura 5.8).

Condición	Prediccion_Linguistica
x6 es ALTO	Prediccion_Linguistica = MEDIA
x6 es MEDIO	Prediccion_Linguistica = MUY ALTA
x6 es MUY ALTO	Prediccion_Linguistica = MUY BAJA
x6 es MUY BAJO	Prediccion_Linguistica = BAJA
x4 es MUY BAJO Y x6 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x4 es ALTO Y x6 es MUY BAJO	Prediccion_Linguistica = MUY BAJA
x6 es BAJO	Prediccion_Linguistica = ALTA
x4 es MUY BAJO Y x6 es MUY ALTO	Prediccion_Linguistica = MUY BAJA

Figura 5.8: Reglas generadas por Fuzzy-LIME con otro dataset

De nuevo, este conjunto de reglas cumple todas las propiedades que debe de tener un modelo de inferencia difusa, a excepción de la completitud. Podemos ver como la variable $x4$ no cubre todas las etiquetas lingüísticas.

El resultado es un diccionario de reglas altamente interpretable que muestra la auténtica naturaleza del conjunto de datos:

- El sistema descarta por completo las variables x_1, x_2, x_3 y x_5. La base de reglas se construye exclusivamente en torno a x_6 y x_4. Esto demuestra que Fuzzy-LIME es capaz de identificar los componentes con mayor impacto general ($7x_6^4$ y $7x_4^1$) frente al resto del polinomio.

- A diferencia de los coeficientes estáticos de LIME, el conjunto de reglas captura perfectamente la curva de decisión de x_6. El sistema detecta que si x_6 es "MUY ALTO", la predicción es "MUY BAJA" (regla 2); sin embargo, en zonas centrales ("MEDIO"), la predicción es "MUY ALTA" (regla 1).

- Por último, el sistema detecta de forma autónoma la sinergia entre las dos variables principales. Por ejemplo, las reglas 4 y 5 muestran cómo la variable x_4 ajusta la predicción cuando x_6 se encuentra en valores bajos, reflejando correctamente el componente lineal $+7x_4^1$ de la ecuación original.

En conclusión, mientras SP-LIME fragmenta la comprensión global en múltiples explicaciones locales que el usuario debe agregar mentalmente, Fuzzy-LIME unifica el comportamiento

del modelo en una única base de conocimiento lingüística. Este enfoque no solo se ajusta a las fronteras de decisión, sino que reduce la carga cognitiva al nivel de una simple lectura de reglas lógicas.

5.1.4. Evaluación con un dataset real

Para validar la aplicabilidad de Fuzzy-LIME en el mundo real, vamos a evaluar el sistema utilizando un conjunto de datos real. Para ello, se seleccionó el *California Housing Dataset*, un problema de regresión clásico en *Machine Learning*.

El objetivo de este conjunto de datos es predecir el valor medio de la vivienda (en cientos de miles de dólares) en distintos distritos de California, utilizando para ello características como el ingreso medio de los residentes (*MedInc*), la antigüedad media de los edificios (*HouseAge*), el número promedio de habitaciones (*AveRooms*) o la población de la zona (*Population*).

Evaluación a Nivel Local

Para comenzar con la evaluación a nivel local en este entorno real, analizamos la instancia 0 del conjunto de test. Para esta vivienda concreta, el modelo de caja negra obtiene una predicción de 4,45 (lo que equivale a un valor de 445.000 dólares). A continuación veremos cómo ambas herramientas explican esta decisión.

Análisis con LIME original

Al observar la explicación generada por el algoritmo LIME estándar (véase la Figura 5.9), de nuevo, comprobamos que el método devuelve un modelo lineal compuesto por ocho coeficientes numéricos. LIME identifica que el nivel de ingresos (*MedInc*, con un peso de +0,69) es el factor que más empuja el precio al alza, mientras que la baja ocupación promedio (*AveOccup*, con −0,52) y las coordenadas geográficas (*Latitude* y *Longitude*, con −0,36 y −0,24 respectivamente) empujan la predicción hacia abajo.

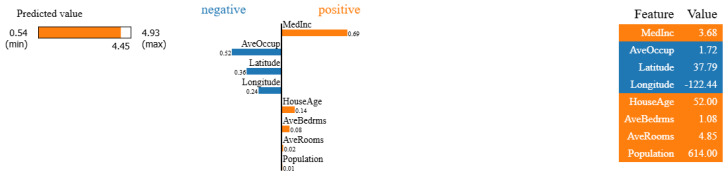


Figura 5.9: Explicación de LIME sobre la instancia 0 del dataset real

Aunque esta información es correcta desde el punto de vista local, volvemos a encontrarnos con el problema de la sobrecarga cognitiva. Para entender el porqué del precio de esta casa, nos vemos obligados a analizar mentalmente ocho variables distintas con valores diferentes (como una latitud de 37,79 frente a una ocupación de 1,72), lo cual dificulta extraer una conclusión rápida e intuitiva.

Análisis con Fuzzy-LIME

Por el contrario, al aplicar nuestra propuesta Fuzzy-LIME sobre la misma instancia (Figura 5.10), el sistema transforma este conjunto de números en un razonamiento lógico directo. El motor difuso clasifica la predicción con la etiqueta **"MEDIA"**, obteniendo un valor continuo de 2,50 a partir del centroide.

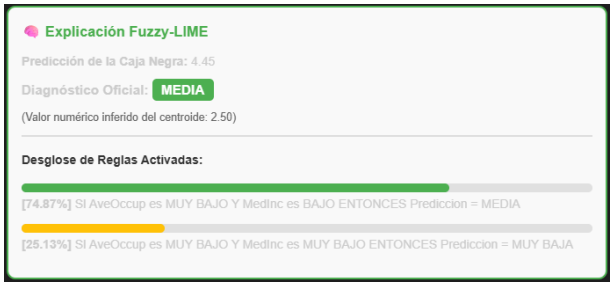


Figura 5.10: Explicación de Fuzzy-LIME sobre la instancia 0 del dataset real

Pese a que la predicción numérica no es tan precisa como la de LIME, lo destacable de este enfoque es cómo el sistema filtra el ruido y aísla la lógica principal (a costa de la precisión numérica), resumiendo el comportamiento en tan solo dos reglas fundamentales:

- **[74.87 %]** SI *AveOccup* es MUY BAJO Y *MedInc* es BAJO ENTONCES Predicción = MEDIA.
- **[25.13 %]** SI *AveOccup* es MUY BAJO Y *MedInc* es MUY BAJO ENTONCES Predicción = MUY BAJA.

Con este resultado, vemos cómo Fuzzy-LIME descarta por completo las variables geográficas y estructurales (antigüedad de la casa, habitaciones, población...) para centrarse exclusivamente en la relación entre la ocupación de la vivienda y los ingresos. Sin embargo esto hace que la predicción numérica del modelo pierda fidelidad, pese a ganar en interpretabilidad ya que la predicción lingüística es razonable para el valor que predijo el modelo de caja negra. Para poder solucionar este problema, se podría ajustar la poda por soporte para obtener más reglas que se ajusten al resultado de esta instancia concreta, pero añadiríamos una complejidad mayor a la interpretabilidad de las reglas a nivel global.

Evaluación a Nivel Global

Ahora vamos a evaluar ambas herramientas para proporcionarnos una visión global del modelo entrenado con el *California Housing Dataset*. El objetivo aquí es comprender qué reglas generales sigue la caja negra para tasar las viviendas en todo el estado.

Análisis con SP-LIME

Para la evaluación global estándar, utilizamos de nuevo el algoritmo *Submodular Pick* (SP-LIME) configurado para seleccionar cinco instancias representativas (véase la Figura 5.11). Al observar los resultados, detectamos una consistencia notable en los pesos: en las cinco explicaciones, el nivel de ingresos (*MedInc*) aparece como el factor positivo principal, mientras que la ocupación promedio (*AveOccup*) y la ubicación geográfica (*Latitude* y *Longitude*) actúan como factores negativos.

Sin embargo, aunque los patrones son claros, nos vemos obligados a inspeccionar y analizar cuarenta coeficientes distintos para confirmar estas tendencias. SP-LIME nos muestra que el modelo es coherente, pero no nos ofrece una regla de decisión unificada, sino que deja en manos del analista la tarea de agregar mentalmente esta información para intuir el funcionamiento global del sistema.

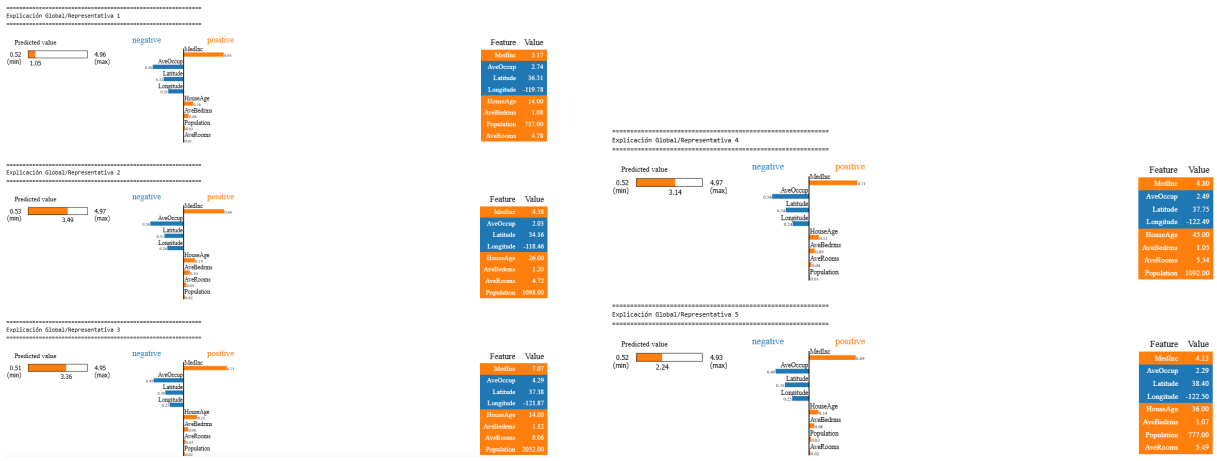


Figura 5.11: Explicación global generada por SP-LIME

Análisis con Fuzzy-LIME

Por su parte, Fuzzy-LIME procesa la base de conocimientos y destila el comportamiento del modelo en un conjunto de once reglas lingüísticas (Figura 5.12). Este enfoque nos permite pasar de observar tendencias numéricas a leer la lógica del modelo directamente.

Condición	Prediccion_Linguistica
AveOccup es MUY BAJO Y HouseAge es MUY BAJO Y MedInc es ALTO	MedHouseVal = MEDIA
AveOccup es MUY BAJO Y MedInc es MUY BAJO	MedHouseVal = MUY BAJA
AveOccup es MUY ALTO Y Latitude es ALTO Y MedInc es MUY BAJO	MedHouseVal = MUY BAJA
AveOccup es MUY BAJO Y HouseAge es ALTO Y MedInc es ALTO	MedHouseVal = MUY ALTA
AveOccup es MUY ALTO Y MedInc es MEDIO	MedHouseVal = BAJA
AveOccup es MUY ALTO Y MedInc es MUY ALTO	MedHouseVal = ALTA
AveOccup es MUY BAJO Y MedInc es BAJO	MedHouseVal = MEDIA
AveOccup es MUY ALTO Y Latitude es MUY BAJO Y MedInc es MUY BAJO	MedHouseVal = MUY BAJA
AveOccup es MUY ALTO Y MedInc es ALTO	MedHouseVal = ALTA
AveOccup es MUY BAJO Y HouseAge es MUY ALTO Y MedInc es ALTO	MedHouseVal = MUY ALTA
AveOccup es MUY ALTO Y Latitude es MUY ALTO Y MedInc es MUY BAJO	MedHouseVal = MUY BAJA

Figura 5.12: Reglas generadas por Fuzzy-LIME con datos reales

De nuevo, este conjunto de reglas cumple todas las propiedades que debe de tener un modelo de inferencia difusa, a excepción de la completitud de la estructura de las reglas.

Sin embargo, al analizar la base de reglas, extraemos conclusiones que con SP-LIME quedarían ocultas:

Podemos ver cómo el sistema combina variables de forma natural. Por ejemplo, vemos que si la ocupación es muy baja (*AveOccup* es MUY BAJO) y los ingresos son altos (*MedInc* es ALTO), el precio varía entre los valores de "MUY ALTA" o "MEDIA" dependiendo de la antigüedad de la casa (*HouseAge*)

Además, las reglas identifican correctamente que cuando los ingresos son muy bajos (*MedInc* es MUY BAJO), la predicción cae a "MUY BAJA" de forma casi invariable, independientemente de otros factores.

De nuevo, observamos el compromiso entre precisión y claridad. Mientras que SP-LIME nos da la precisión de cinco modelos lineales locales, Fuzzy-LIME nos ofrece un manual de instrucciones lingüístico. Aunque este último simplifica ciertos matices numéricos, nos proporciona una

comprensión del "porqué" global mucho más fácil y rápida de asimilar, permitiéndonos comprender la lógica del modelo de caja negra de una manera mucho más cercana al razonamiento experto.

5.2. Evaluación de FuzzyRank-LIME

En esta sección se realiza la validación de la arquitectura **FuzzyRank-LIME**. El objetivo de esta sección es demostrar cómo la traducción de los pesos numéricos a un dominio lingüístico proporciona una mayor robustez semántica y mitiga la carga cognitiva ligada a los coeficientes lineales de LIME. Este análisis de resultados se ha estructurado en dos perspectivas complementarias: por un lado, se realizará el análisis de los resultados obtenidos al utilizar este enfoque con los datos sintéticos originales y, por otro lado, se realizará un análisis similar con los resultados de utilizar el dataset *California Housing Dataset*.

5.2.1. Resultados con el dataset original

En primer lugar, empezaremos analizando los resultados obtenidos con el dataset original. Este análisis de resultados se ha dividido a su vez en dos perspectivas complementarias: en primer lugar, se realizará una evaluación a nivel local, detallando la fuzzificación de los pesos de importancia para una instancia aislada del conjunto de datos y, en segundo lugar, se abordará la evaluación a nivel global, integrando FuzzyRank-LIME con SP-LIME para abstraer el comportamiento general del modelo.

Evaluación a Nivel Local

Para validar el enfoque de **FuzzyRank-LIME** a nivel local, se analiza la **instancia 0** del conjunto de test. En esta fase, el objetivo no es extraer reglas lógicas, sino transformar los pesos numéricos generados por las múltiples iteraciones de LIME en etiquetas semánticas que capturen la incertidumbre del modelo. Para este experimento local, se ha configurado el coeficiente de difusividad o *fuzziness* en $m = 3$. Este valor permite obtener funciones de pertenencia más estrechas y con menor solapamiento (véase la Figura 5.13).

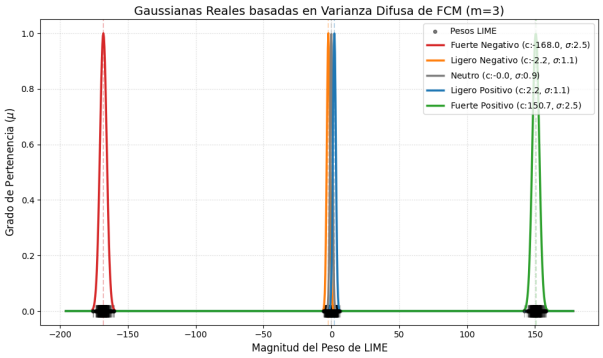


Figura 5.13: Conjuntos difusos de la importancia en una instancia concreta

Análisis de los Grados de Pertenencia y Pesos de Importancia

Los resultados obtenidos tras el proceso de fuzzificación de los pesos medios originales se pueden observar en la Figura 5.14. La principal ventaja de FuzzyRank-LIME es su capacidad para gestionar la ambigüedad de las variables cuyo peso es cercano a los umbrales de decisión entre etiquetas:

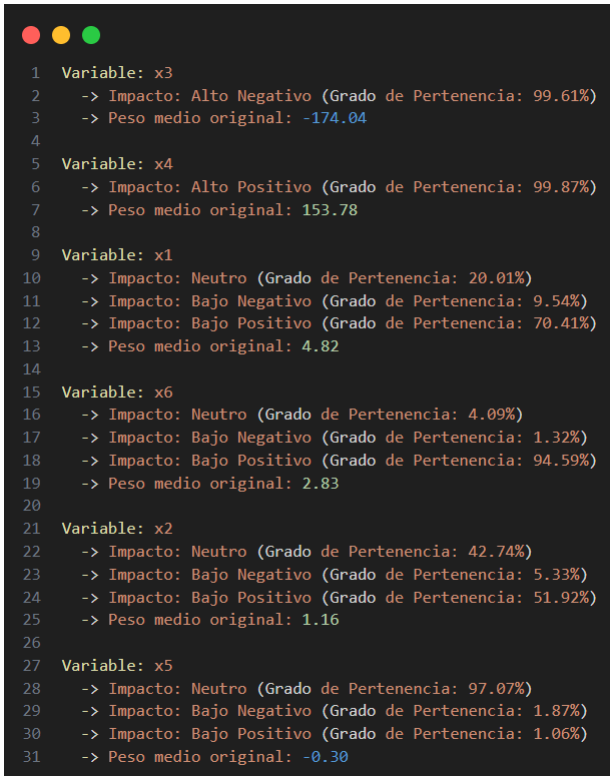


Figura 5.14: Importancia de cada variable en una instancia concreta

Como se puede ver, las variables x_3 y x_4 tienen las importancias más altas. La variable x_3, con un peso medio de $-174{,}04$, es clasificada como **"Alto Negativo"** con un grado de pertenencia del 99,61 %. De igual manera, x_4 (153,78) se asigna a **"Alto Positivo"** con un 99,87 %. En estos casos, la magnitud del peso es tan clara que el sistema elimina cualquier duda al usuario.

El valor de FuzzyRank-LIME destaca en variables como x_1 (4,82) y x_2 (1,16). En LIME estándar, el analista debería decidir si un 1,16 es relevante o no. En nuestro enfoque, el sistema reconoce la ambigüedad: x_2 pertenece en un 51,92 % a **"Bajo Positivo"** pero también en un 42,74 % a **"Neutro"**. Esta dualidad refleja la inestabilidad del modelo en esa zona, evitando conclusiones erróneas.

La variable x_5, con un peso casi nulo ($-0{,}30$), es asignada directamente a la etiqueta **"Neutro"** con un 97,07 %. Esto permite al usuario ignorar visualmente esta característica en la explicación final, centrándose solo en las etiquetas con significado semántico importante.

Conclusiones sobre la Visualización Lingüística

La Figura 5.13 muestra cómo las campanas gaussianas, ajustadas con $m = 3$, cubren el espectro de pesos. Gracias a este ajuste, la variable x_6, a pesar de tener un peso numérico bajo (2,83), logra una pertenencia del 94,59 % a **"Bajo Positivo"**.

Este análisis demuestra que FuzzyRank-LIME proporciona una nueva capa de abstracción: en lugar de obligar al usuario a interpretar seis valores decimales con signos distintos, el sistema devuelve una salida donde se identifica inmediatamente que la predicción está dominada por una fuerte oposición entre x_3 y x_4, con contribuciones menores de x_1, x_6 y x_2.

Evaluación a Nivel Global

Para evaluar la capacidad de síntesis global de la arquitectura **FuzzyRank-LIME**, se ha integrado el sistema con el algoritmo *Submodular Pick* (SP-LIME). En este escenario, el objetivo es doble: por un lado, seleccionar las $K = 5$ instancias más representativas del modelo y, por otro, transformar sus pesos numéricos en una salida lingüística que permita identificar patrones de comportamiento globales. Para esta configuración, se ha empleado un coeficiente de *fuzzyness* $m = 2$, lo que genera funciones de pertenencia ligeramente más anchas y suaves (véase la Figura 5.15), optimizando la transición entre categorías semánticas en un entorno de mayor variabilidad de datos.

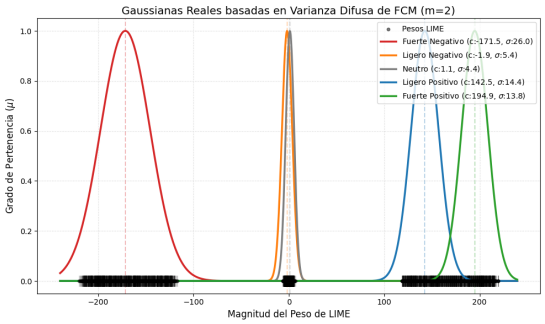


Figura 5.15: Conjuntos difusos de la importancia desde un enfoque global

Análisis de Estabilidad en Instancias Representativas

Los resultados que vemos en la Figura 5.16 muestran una consistencia lógica a lo largo de las cinco explicaciones representativas. A pesar de que los pesos numéricos originales varían debido a la naturaleza local de cada punto de muestreo, la traducción lingüística permite extraer conclusiones globales sobre el modelo de caja negra:

En primer lugar vemos como el sistema identifica a x_3 y x_4 como las variables más importantes. Es importante observar que, aunque el peso de x_3 varía desde $-122{,}28$ (Explicación 3) hasta $-212{,}00$ (Explicación 1), FuzzyRank-LIME clasifica ambos valores de forma estable como **"Alto Negativo"**, con altos grados de pertenencia. Lo mismo sucede con x_4, que se mantiene mayoritariamente en la categoría **"Alto Positivo"**, exceptuando zonas específicas donde su impacto se suaviza a **"Bajo Positivo"** (Explicación 5).

Las variables x_1, x_2, x_5 y x_6 son clasificadas consistentemente bajo la etiqueta **"Neutro"** o, en su defecto, presentan impactos residuales de tipo **"Bajo Negativo/Positivo"**. Esta convergencia global indica que, para la mayoría de las decisiones del modelo, estas características no suponen una influencia determinante, permitiendo al analista simplificar el resumen del sistema.

Gracias al uso de la lógica difusa, el sistema es capaz de señalar cuándo una variable se encuentra en una zona de transición. Un ejemplo claro se observa en la Explicación 1 para la variable x_4 (171,78), donde el sistema detecta una pertenencia compartida entre **"Alto Positivo"** (60,07 %) y **"Bajo Positivo"** (37,49 %).

Síntesis de la Interpretabilidad Global

La principal aportación de FuzzyRank-LIME en este nivel es la simplificación de la base de conocimiento. Como se observa en el análisis de las cinco instancias, FuzzyRank-LIME logra lo que SP-LIME por sí solo no puede: proporcionar un lenguaje común. Mientras que SP-LIME nos obligaría a comparar cinco tablas de coeficientes distintos, FuzzyRank-LIME nos devuelve cinco

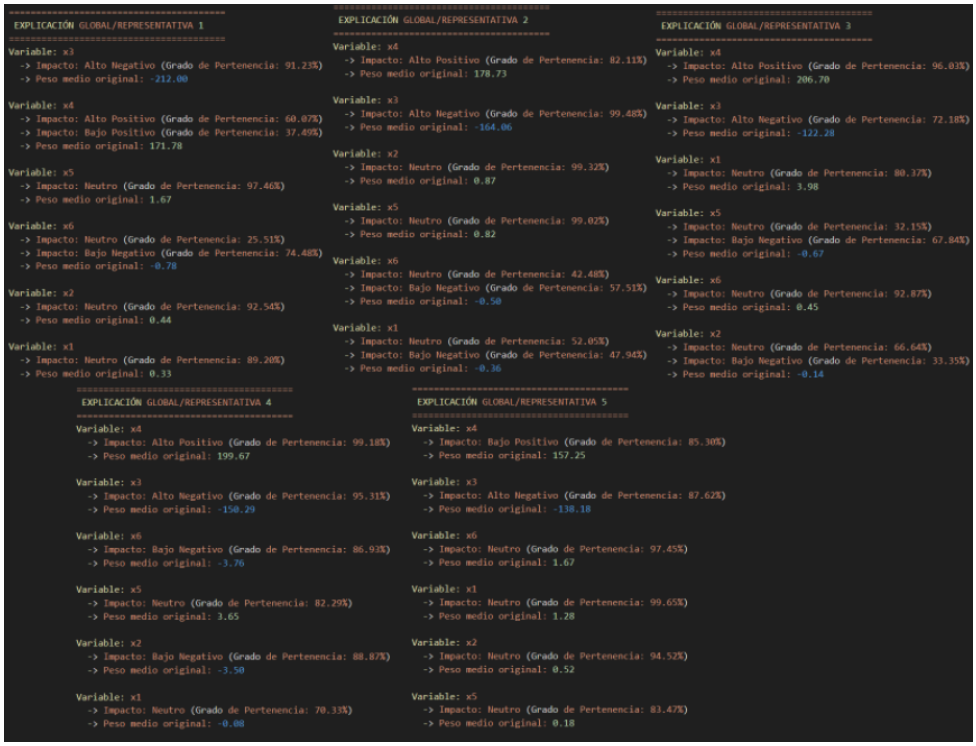


Figura 5.16: Importancia de cada variable en 5 instancias representativas (SP-LIME)

salidas lingüísticas comparables, confirmando que la lógica del modelo es robusta y se basa, en su mayoría, en la oposición entre un fuerte impacto negativo de x_3 y un fuerte impacto positivo de x_4.

5.2.2. Resultados sobre un dataset real

Con el propósito de comprobar el comportamiento y la utilidad de FuzzyRank-LIME en un escenario práctico, aplicamos de nuevo el *California Housing Dataset*. A continuación, evaluamos la capacidad de la herramienta para transformar la incertidumbre matemática en conocimiento lingüístico, dividiendo el análisis en una perspectiva local y otra global.

Evaluación a Nivel Local

Para evaluar el comportamiento de FuzzyRank-LIME a nivel local en un entorno real, analizamos la instancia 0 del conjunto de prueba. En este experimento, configuramos el coeficiente de *fuzzyness* en $m = 2$, lo que nos permite generar funciones de pertenencia (campanas de Gauss) con un nivel de solapamiento adecuado (véase la Figura 5.17).

Al aplicar el proceso de fuzzificación sobre los pesos, obtenemos las siguientes importancias para cada variable (Figura 5.18). A diferencia de la interpretación de coeficientes lineales, este enfoque nos muestra la toma de decisiones del modelo.

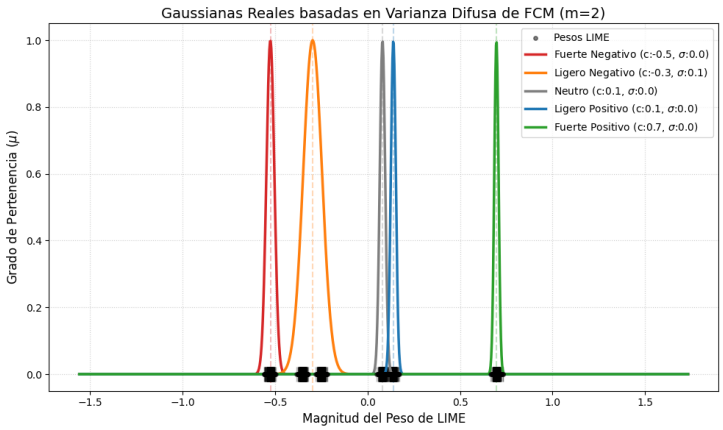


Figura 5.17: Conjuntos difusos de la importancia en una instancia concreta del dataset real

```
Variable: MedInc
  -> Impacto: Alto Positivo (Grado de Pertenencia: 99.91%)
  -> Peso medio original: 0.69

Variable: AveOccup
  -> Impacto: Alto Negativo (Grado de Pertenencia: 99.76%)
  -> Peso medio original: -0.54

Variable: Latitude
  -> Impacto: Bajo Negativo (Grado de Pertenencia: 94.53%)
  -> Impacto: Alto Negativo (Grado de Pertenencia: 3.90%)
  -> Peso medio original: -0.34

Variable: Longitude
  -> Impacto: Bajo Negativo (Grado de Pertenencia: 94.98%)
  -> Impacto: Neutro (Grado de Pertenencia: 1.48%)
  -> Impacto: Bajo Positivo (Grado de Pertenencia: 1.07%)
  -> Impacto: Alto Negativo (Grado de Pertenencia: 2.28%)
  -> Peso medio original: -0.26

Variable: HouseAge
  -> Impacto: Neutro (Grado de Pertenencia: 8.13%)
  -> Impacto: Bajo Positivo (Grado de Pertenencia: 91.29%)
  -> Peso medio original: 0.16

Variable: AveBedrms
  -> Impacto: Neutro (Grado de Pertenencia: 99.61%)
  -> Peso medio original: 0.08
```

Figura 5.18: Importancia de cada variable en una instancia concreta del dataset real

- Por un lado, el sistema clasifica el nivel de ingresos (*MedInc*, con un peso medio de 0,69) como un impacto **"Alto Positivo"** con un grado de pertenencia casi total (99,91 %). Por el contrario, la ocupación promedio (*AveOccup*, −0,54) se clasifica como **"Alto Negativo"** (99,76 %). La alta pertenencia en ambos casos nos indica que el modelo no tiene ninguna duda sobre la importancia de estas variables para esta vivienda.

- Además, destaca el comportamiento de las variables geográficas y la antigüedad de la casa. Aunque la latitud (−0,34) y la longitud (−0,26) tienen coeficientes numéricos distintos, FuzzyRank-LIME agrupa ambos impactos bajo el mismo concepto semántico: **"Bajo Negativo"** (con 94,53 % y 94,98 % de pertenencia, respectivamente). De igual forma, la antigüedad de la vivienda (*HouseAge*, 0,16) se mapea como un impacto **"Bajo Positivo"** (con un 91,29 %).

- Por último, la variable número de habitaciones (*AveBedrms*), cuyo peso medio es de apenas 0,08, corresponde casi totalmente a la etiqueta **"Neutro"** (99,61 %). Esto permite al usuario descartarla visualmente del análisis de esta predicción.

Conclusiones sobre el Perfil Lingüístico Local

Este análisis muestra la principal ventaja de FuzzyRank-LIME en entornos reales: la **agrupación semántica**. El algoritmo LIME tradicional nos forzaría a evaluar la diferencia numérica entre el −0,34 de la latitud y el −0,26 de la longitud. Sin embargo, FuzzyRank-LIME comprende que ambos valores representan el mismo nivel de impacto (Bajo Negativo).

Evaluación a Nivel Global

Para concluir el análisis de FuzzyRank-LIME sobre este conjunto de datos reales, evaluamos su integración con el algoritmo de selección SP-LIME. Configuramos de nuevo el sistema para extraer cinco instancias representativas y mantenemos el coeficiente de *fuzzyness* en m=2 (véase la Figura 5.19).

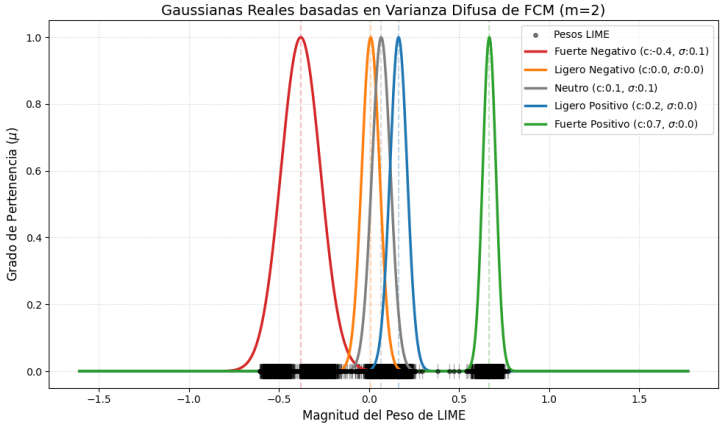


Figura 5.19: Conjuntos difusos de la importancia desde un enfoque global con el dataset real

Al observar los grados de pertenencia generados para estas cinco explicaciones (Figura 5.20), comprobamos que FuzzyRank-LIME actúa como un gran unificador semántico. Aunque los coeficientes numéricos originales varían de una instancia a otra debido a la naturaleza local del muestreo, la traducción a lenguaje difuso nos muestra que el modelo subyacente obtiene explicaciones bastante estables.

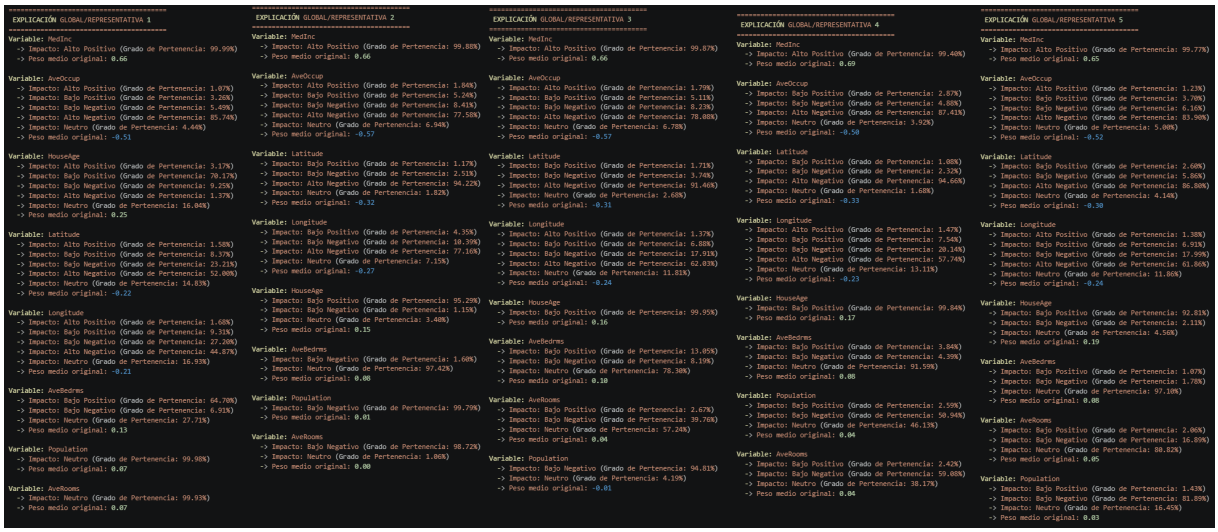


Figura 5.20: Importancia de cada variable en 5 instancias representativas del dataset real

- En las cinco explicaciones generadas, el nivel medio de ingresos (*MedInc*) es la variable más importante. A pesar de ligeras variaciones en su peso numérico (entre 0.65 y 0.69), el sistema lo clasifica como **"Alto Positivo"** con grados de pertenencia que rozan el 100 %. Por otro lado, la ocupación (*AveOccup*) se etiqueta siempre como **"Alto Negativo"** (superando el 77 % de pertenencia en todos los casos).

- Mientras que en LIME estándar el analista debe deducir la tendencia de variables secundarias, FuzzyRank-LIME nos muestra de forma directa que la latitud y la longitud tienen un impacto **"Alto Negativo"**. Lo mismo ocurre con la antigüedad de la casa (*HouseAge*) que tiene un nivel de importancia **"Bajo Positivo"** en todas las instancias (alcanzando pertenencias superiores al 90 % en la mayoría de los casos).

- Por último, el sistema nos permite descartar el resto de variables. Características como la población (*Population*) o el número de habitaciones (*AveRooms* y *AveBedrms*) cuentan en su mayoría con la etiqueta **"Neutro"** o presentan una alta ambigüedad entre un impacto nulo y uno muy bajo. Esto nos indica que no necesitamos incluirlas para entender cómo funciona el algoritmo.

5.3. Evaluación de SC-LIME

En esta sección se analiza el desempeño de la arquitectura **SC-LIME**. Con el fin de evaluar la adaptabilidad de la herramienta a diferentes conjuntos de datos, se presentan los resultados obtenidos sobre dos *datasets* sintéticos con niveles de complejidad distintos, además del dataset real *California Housing Dataset*. Este análisis comparativo permite observar cómo el algoritmo ajusta el número de prototipos necesarios para explicar el modelo, evitando la redundancia informativa y optimizando la comprensión global del sistema.

5.3.1. Análisis sobre el Conjunto de Datos Original

El primer experimento se realizó sobre el conjunto de datos original, cuya lógica de decisión está dominada por un comportamiento lineal entre sus variables principales. Al aplicar el algoritmo de *Subtractive Clustering* sobre el espacio de explicaciones, se observa una síntesis de información notable que diferencia esta propuesta de la implementación estándar.

A diferencia del algoritmo SP-LIME convencional, que fue forzado a seleccionar $K = 5$ instancias representativas independientemente de la redundancia entre ellas, SC-LIME identifica de forma autónoma que una única instancia es suficiente para representar fielmente el comportamiento global del modelo en este escenario. El algoritmo detecta un único pico de densidad en el espacio de los pesos, indicando que el modelo de caja negra aplica una lógica consistente y uniforme en todo el espacio de características.



Figura 5.21: Explicación global generada por SC-LIME con el conjunto de datos del polinomio conocido

Como se observa en la Figura 5.21, la instancia seleccionada captura con gran precisión el núcleo del modelo.

Como podemos ver, el sistema identifica a x_3 (con peso: $-168{,}43$) y x_4 (con peso: $+166{,}60$) como las únicas variables importantes de la predicción. La magnitud de estos pesos, en comparación con el resto de variables, muestra una estructura de decisión clara y balanceada.

Las variables x_1, x_2, x_5 y x_6 presentan pesos residuales (entre 0,72 y 1,82), lo que confirma que el algoritmo de clustering ha seleccionado una instancia donde el ruido no enmascara la lógica principal.

Con un valor predicho de $-2{,}77$ frente a un rango de salida de $[-718{,}97, 715{,}04]$, la instancia seleccionada representa un punto de equilibrio donde la interacción entre la fuerte carga negativa de x_3 y la positiva de x_4 se hace evidente.

Este resultado demuestra la eficiencia de SC-LIME: mientras que SP-LIME obligaría al analista a revisar cinco (o N) tablas de datos prácticamente idénticas, nuestra propuesta simplifica la interpretabilidad a una única explicación que contiene toda la información necesaria para comprender su funcionamiento global.

5.3.2. Análisis sobre el nuevo Conjunto de Datos

El segundo experimento con SC-LIME se ejecutó sobre el conjunto de datos generado por la función polinómica no lineal (Ecuación 5.1), donde destacan los términos $+7x_4^1$ y $-7x_6^4$. El propósito de esta prueba es evaluar si el algoritmo basado en densidad es capaz de escalar su complejidad cuando se enfrenta a fronteras de decisión menos definidas.

De manera autónoma, y contrastando con el experimento anterior donde convergió a un único prototipo, SC-LIME ha determinado que el comportamiento global de este modelo se explica de manera óptima mediante 3 instancias representativas. Esto supone una mejora analítica frente a SP-LIME (que, por diseño, había sido forzado a extraer 5 explicaciones locales, introduciendo redundancia). El algoritmo de clustering ha detectado tres centros de masa en el espacio de importancias locales.

Al analizar las tres explicaciones extraídas (Figura 5.22), se muestra una captura perfecta del funcionamiento del modelo subyacente.

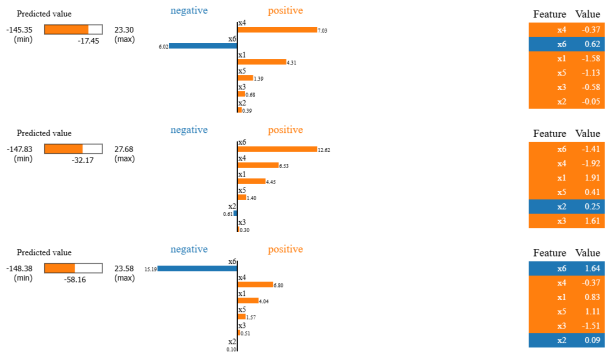


Figura 5.22: Explicación global generada por SC-LIME con otro conjunto de datos

En las tres explicaciones seleccionadas, la variable x_4 presenta un peso positivo estable (7,03, 6,53 y 6,80, respectivamente). SC-LIME ha identificado correctamente que, independientemente

de la región del espacio en la que nos encontremos, la variable x_4 siempre aporta un empuje positivo constante. Esto es un reflejo del término lineal $+7x_4$ de la ecuación original.

El mayor logro explicativo de SC-LIME en este experimento es cómo ha seleccionado estratégicamente tres puntos en la curva generada por el término $-7x_6^4$. Dado que los pesos de LIME representan el gradiente local (la pendiente), SC-LIME ha encontrado los tres comportamientos posibles de esta curva:

Explicación 1 (con $x_6 = 0{,}62$): Un valor positivo moderado, donde la caída de la curva empieza a notarse, generando una importancia negativa moderada (peso: $-6{,}02$).

Explicación 2 (con $x_6 = -1{,}41$): Un valor negativo alto. Dado el exponente par, el valor de x_6^4 es alto, pero al acercar x_6 hacia el cero, el valor de la función aumenta. SC-LIME capta esta pendiente ascendente asignándole un peso bastante positivo (peso: $+12{,}62$).

Explicación 3 (con $x_6 = 1{,}64$): Un valor positivo alto. Aquí la curva cae en picado, lo que se refleja en una importancia negativa muy alta (peso: $-15{,}19$).

Este análisis confirma la superioridad de SC-LIME para la explicabilidad global. En lugar de forzar una diversidad artificial basada en la aparición de características (como hace SP-LIME), la selección por *Subtractive Clustering* mapea las decisiones reales del modelo.

SC-LIME ha demostrado que 5 instancias eran demasiadas (generando ruido) y 1 era insuficiente (incapaz de mostrar una curva). Al seleccionar exactamente 3 explicaciones, el sistema ofrece al usuario la cantidad mínima de información necesaria para deducir mentalmente que el modelo contiene una constante lineal (x_4) y una curva de orden superior (x_6), maximizando así la fidelidad en las explicaciones y minimizando la carga de la interpretabilidad.

5.3.3. Análisis sobre un Conjunto de Datos real

Por último, vamos a evaluar la arquitectura SC-LIME sobre el mismo conjunto de datos reales (*California Housing Dataset*). Al analizar anteriormente los resultados de SP-LIME con este conjunto de datos, identificamos un patrón muy claro y repetitivo en sus cinco instancias representativas: el modelo siempre premiaba el nivel de ingresos (*MedInc*) y penalizaba la alta ocupación (*AveOccup*) y ciertas coordenadas geográficas.

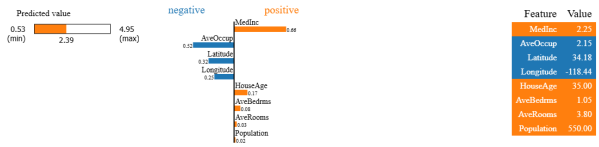


Figura 5.23: Explicación global generada por SC-LIME con el conjunto de datos real

Como podemos observar, SC-LIME llega otra vez a la conclusión de que una única instancia es suficiente para representar el comportamiento global del modelo en este *dataset*.

Al evaluar los resultados de esta única explicación (Figura 5.23), comprobamos que la instancia seleccionada captura el comportamiento del modelo que intuíamos anteriormente:

- Vemos un impacto positivo dominante de los ingresos (*MedInc*, con un peso de +0.66), que es contrarrestado casi en la misma medida por el impacto negativo de la ocupación de la vivienda (*AveOccup*, con -0.52).

- Además, el algoritmo selecciona una instancia donde la latitud (-0.32) y la longitud (-0.25) muestran su peso real como penalizadores del precio, consolidando el patrón que SP-LIME nos obligaba a buscar a lo largo de varias explicaciones.
- Por último, variables como la antigüedad de la casa (*HouseAge*, +0.17) o el número de habitaciones (*AveBedrms*, +0.08) se quedan en un segundo plano, confirmando que su impacto es menor a nivel global.

Este resultado es muy útil para justificar la utilidad de SC-LIME en un problema real. El algoritmo nos demuestra que, si un modelo de caja negra aplica una lógica consistente de forma generalizada para tasar viviendas, no tiene sentido mostrar al usuario múltiples ejemplos similares. Al extraer este único "centro de masa", obtenemos la misma comprensión global que con SP-LIME, pero reducimos la carga cognitiva y confirmamos que la herramienta es capaz de detectar y eliminar la redundancia incluso en escenarios reales.

Capítulo 6

Conclusiones y vías futuras

En este trabajo hemos reivindicado el papel del *Soft Computing* y la Lógica Difusa en esta nueva era de la IA dominada por el aprendizaje profundo de modelos de "caja negra" y donde ha surgido, como consecuencia, la disciplina conocida como Inteligencia Artificial Explicable (XAI), cuyo fin último es arrojar luz sobre las decisiones tomadas por estos modelos.

La revisión de diferentes técnicas procedentes tanto de *Soft Computing* como de XAI, junto con la investigación sobre el estado del arte nos ha llevado a la exploración de diferentes combinaciones de las mismas, y nos ha servido para realizar una prueba de concepto que nos pone en el camino adecuado para la consecución de nuestro objetivo. Así, hemos explorado combinaciones de LIME (Local Interpretable Model-agnostic Explanations), puesto que es una de las herramientas más usadas en el ámbito de la XAI y no cuenta con una implementación que lo combine con *Soft Computing*, con tres técnicas de *Soft Computing*, como lo son los árboles de decisión y métodos de *fuzzy clustering*, las cuales son los enfoques más utilizados a la hora de obtener sistemas de inferencia difusos a partir de datos: **Fuzzy-LIME**, que sustituye el modelo de "caja blanca" lineal de LIME por un sistema de reglas difusas obtenido a partir de árboles de decisión; **FuzzyRank-LIME**, que genera el ranking como LIME con conjuntos difusos con etiquetas lingüísticas en lugar de números reales; **SC-LIME**, que usa *Subtractive Clustering* para seleccionar un conjunto de instancias representativas para explicar el comportamiento global del modelo de "caja negra". Adicionalmente, como consecuencia del enfoque Fuzzy-LIME, se ha desarrollado una API configurable por el usuario la cual podría extenderse en el futuro para incluir nuevas combinaciones y ajustes de parámetros.

Estas implementaciones muestran que la hibridación de estos paradigmas ayuda a superar las limitaciones de rigidez e inestabilidad del LIME tradicional dado que los conjuntos difusos con etiquetas lingüísticas son más descriptivos que los números reales y las funciones de pertenencia absorben la variabilidad de los resultados de LIME. Estos resultados son indicadores de que la relación entre el *Soft Computing* y la XAI es **sinérgica**.

Además, si añadiéramos nuestros enfoques como una nueva línea de la tabla que resume el trabajo de análisis realizado en [PAT25], en esta se marcarían con un tick las características de *Feature Generation* y *Sample Generation* y podría presentar como problemas Fidelity Issue (F) y Efficiency Issue (E).

Como vías futuras, en primer lugar, se podría extender la API de Fuzzy-LIME para integrar nuestros tres enfoques de modo que el usuario decida cuál quiere utilizar. Por otro lado, proponemos que se usen otras técnicas adecuadas para la obtención de un Sistema de Inferencia Difuso desde datos como la integración de *Subtractive Clustering* con *Fuzzy C-Means* y *ANFIS* o como

la hibridación propuesta en el artículo [MER20] que en lugar de usar *Fuzzy C-Means* utiliza *Gustafson-Kessel Clustering* para detectar una serie de reglas difusas, las cuales se amoldan más a la estructura de los datos que con *Fuzzy C-Means* el cual impone una estructura, e incluirlos como nuevos enfoques en dicha API.

Por otro lado, esta idea se podría ampliar para adaptar los mecanismos de fuzzificación y extracción de reglas para operar sobre datos no tabulares, como imágenes o texto.

Finalmente, nuestros enfoques se podrían utilizar en aquellas áreas donde ya se utiliza la XAI, aportando la solución a los problemas de rigidez e inestabilidad de LIME. Esto permitirá la reducción de la carga cognitiva y el incremento de la confianza que nuestras herramientas aportan frente a las explicaciones lineales tradicionales.

Bibliografía

[ALE25] Alexandra I. Khalyasmaa, Pavel V. Matrenin y Stanislav A. Eroshenko (2025). *Interpretable Diagnostics with SHAP-Rule: Fuzzy Linguistic Explanations from SHAP Values.*, 21-10-2025.

[BAB96] Robert Babuška (1996). *Fuzzy Modeling and Identification*, 1996.

[CHI94] S.L. Chiu (1994). *Fuzzy Model Identification Based on Cluster Estimation*, Journal of Intelligent and Fuzzy Systems Vol. 2, 267-278, 23-06-1994.

[EDA25] Eda Kavlakoglu (2025). *¿Qué es un árbol de decisión?*, IBM, 27-11-2025

[IEEE1] IEEE Xplore, Shijun Tang et al. (2024). *Manipulability Optimization of Omnidirectional Mobile Redundant Manipulator Considering Obstacle Avoidance With Fuzzy Adaptive Controller*, 2024.

[IEEE2] IEEE Xplore, Wei Liu et al. (2024). *Fuzzy Compounded State Observer-Based Finite-Time Preassigned Performance Control for Nonlinear Systems With Asymmetric Time-Varying State Constraints*, 2024.

[IEEE3] IEEE Xplore, Wenchuan Zang et al. (2024). *Angle of Attack Enhanced Trajectory Prediction for Underwater Gliders Based on Fuzzy-Fusion of Inference and Dynamics*, 2024.

[IEEE4] IEEE Xplore, Xinran Zhou et al. (2025). *A Novel Multigranularity Clustering Algorithm Based on Grid Partition and Fuzzy Quotient Space*, 2025.

[IEEE5] IEEE Xplore, Rui Hou et al. (2025). *CAE-FCM: Context-Aware Enhanced Fuzzy Cognitive Maps for Interpretable Multivariate Time Series Forecasting*, 2025.

[IEEE6] IEEE Xplore, Meng-Wei Li et al. (2025). *A Linguistically Interpretable Fuzzy Fault Diagnosis Model: Knowledge Distillation Perspective*, 2025.

[JOR26] Jorge Montesinos Bravo (2026). *TFM Explainable Fuzzy AI*, https://github.com/montesinos3/TFM, 28-05-2026.

[JOS25] Jose Manuel Juarez Herrero (2025). *MACHINE LEARNING EXPLICABLE MODULE 3. XAI METHODS (part II)*, Universidad de Murcia, 2025.

[LUN17] Scott M. Lundberg y Su-In Lee (2017). *A Unified Approach to Interpreting Model Predictions*, University of Washington, 25-11-2017.

[MER03] Mercedes Valdés Vela. *Definición de un Marco para la Especificación de Técnicas Híbridas de mOdelado Difuso (METHOD)*, Universidad de Murcia, Noviembre de 2003.

[MER20] Mercedes Valdés Vela, Jesús Cuenca-Jara, Fernando Terroso-Sáenz y Antonio Skarmeta (2020). *Classification of spatio-temporal trajectories from Volunteer Geographic Information through fuzzy rules*, Universidad de Murcia, Enero de 2020.

[MER25a] Mercedes Valdés Vela y José Tomás Palma Méndez (2025). *Tema 3: Conocimiento y Razonamiento Aproximado Basado en Lógica Difusa. Parte I: Conjuntos Difusos*, Universidad de Murcia, 2025.

[MER25b] Mercedes Valdés Vela y José Tomás Palma Méndez (2025). *Tema 3: Conocimiento y Razonamiento Aproximado Basado en Lógica Difusa. Parte II: Lógica Difusa*, Universidad de Murcia, 2025.

[MER25c] Mercedes Valdés Vela y José Tomás Palma Méndez (2025). *Tema 3: Conocimiento y Razonamiento Aproximado Basado en Lógica Difusa. Parte III: Sistemas de Inferencia Difusos*, Universidad de Murcia, 2025.

[MOL19] Christoph Molnar (2019). *Interpretable Machine Learning. A Guide for Making Black Box Models Explainable*, 21-02-2019.

[MUH19] Muhammad Rehman Zafar, Naimul Mefraz Khan (2019). *DLIME: A Deterministic Local Interpretable Model-Agnostic Explanations Approach for Computer-Aided Diagnosis Systems*, Ryerson University Toronto, Canada, 24-06-2019.

[OLI99] J. Valente de Oliveira (1999). *Semantic Constraints for Membership Function Optimization*, IEEE Transactions on Systems, Man, and Cybernetics 19 , no. 1, 128-138, Enero de 1999.

[PAT25] Patrick Knab, Sascha Marton, Udo Schlegel, and Christian Bartelt (2025). *Which LIME should I trust? Concepts, Challenges, and Solutions.*, 2025.

[PAV25] Alexandra I. Khalyasmaa, Pavel V. Matrenin y Stanislav A. Eroshenko (2025). *Interpretable Diagnostics with SHAP-Rule: Fuzzy Linguistic Explanations from SHAP Values*, Russia, 21-10-2025.

[RIB16] Ribeiro, M. T., Singh, S., y Guestrin, C. (2016). *"Why should I trust you?": Explaining the predictions of any classifier.*, 09-08-2016.

[SON23] Sonia Laguna, Julian N. Heidenreich, Jiugeng Sun, Nilüfer Cetin, Ibrahim Al-Hazwani, Udo Schlegel, Furui Cheng, Mennatallah El-Assady (2023). *ExpLIMEable: An exploratory framework for LIME*, ETH Zurich, 2023.

[SUG85] Tomohiro Takagi y Michio Sugeno (1985). *Fuzzy Identification of Systems and Its Applications to Modeling and Control*, Febrero de 1985.

[ZER23] Zeren Tan, Yang Tian y Jian Li (2023). *GLIME: General, Stable and Local LIME Explanation*, Tsinghua University, 2023.

[ZOU21] Zoumpolia Dikopoulou, Serafeim Moustakidis y Patrik Karlsson (2021). *gLIME: A NEW GRAPHICAL METHODOLOGY FOR INTERPRETABLE MODEL-AGNOSTIC EXPLANATIONS*, 22-07-2021.

Uso de la IA

Para la realización de este Trabajo de Fin de Máster se ha hecho uso de herramientas de IA generativa como ChatGPT y Gemini para ayudar a revisar la redacción de este documento.

Apéndice

Para poder ver toda la implementación de las combinaciones realizadas en este proyecto, contamos con un Github público donde se pueden explorar los diferentes *Jupyter Notebooks* [JOR26].